

Ratemaking and Product Information

Recommender Systems for Insurance Marketing

Giorgio Alfredo Spedicato^{1 a}, Giuseppe Savino^{2 b}

¹ Unipol Group, ² Zanichelli Group

Keywords: insurance marketing, predictive modeling, upselling, generalized low rank models, recommender systems, collaborative filtering, supervised learning, matrix factorization, arules

Variance

Vol. 15, Issue 1, 2022

Recommender systems are machine learning algorithms typically employed to support marketing decisions, as they identify statistically validated associations between products and consumers. These tools have been successfully adopted in many fields; however, not much has been done for the insurance industry. The paper aims to fill that gap by giving an overview of various relevant recommender systems, both from a theoretical and a practical perspective. In particular, classical approaches are compared to methods based on machine learning techniques. Finally, we present a comparison of the performance of the reviewed methodologies in the context of an insurance case study. It appears that machine learning-based approaches (such as gradient boosting trees and deep neural networks) generally provide stronger performance at the cost of longer computational times.

1. INTRODUCTION

Recommender systems (RSs) are machine learning algorithms that suggest to loyal clients the best next product to buy or propose to prospective customers the most appropriate set of products (Ricci et al. 2011).

Currently, these algorithms constitute one of the more active research areas of machine learning (ML), partly because of their successful employment in various business sectors, from the entertainment industry to e-commerce. Netflix's movie suggestions (Netflix 2009), Amazon's recommended site sections, and Twitter's "users to follow" are just a few well-known outstanding examples of RS use cases.

However, despite this great popularity, the insurance industry (II) has paid relatively little attention to them. In particular, by mid-2018 only a few articles (Mitra, Chaudhari, and Patwardhan 2014; Rokach et al. 2013; Qazi et al. 2017) in the actuarial literature had focused on the application of RSs to the II. The reasons behind the paucity of scientific papers have mainly to do with the intrinsic nature of the II when compared to other fields, where RSs have been rewardingly applied (Rokach et al. 2013). Building an RS capable of bringing value to both companies and consumers, using canonical techniques, requires either a large set of

similar products to recommend or a vast amount of unwittingly provided profiling data characterizing clients and prospects. In contrast, in the insurance industry the number of products available is relatively small, limiting the effectiveness of classical RS models. Moreover, many techniques adopted in RS analyses are quite different from those routinely used by actuaries. Nevertheless, we believe that unlocking the potential of RSs in the II could probably benefit insurers—that is, by proposing the purchase of nonregulated coverages, which typically offer a higher underwriting margin than regulated ones.

This work aims at providing an overview of the state-of-the-art RS algorithms from theory to code implementation in a real-case scenario, showing that such techniques could find fruitful application in actuarial science. Furthermore, we suggest a way to mathematically model the recommendations in the II field and we build an RS approach based on supervised learning algorithms. For our data, this suggested method shows promising results in overcoming the difficulties of the classical RS models in the II.

2. BRIEF LITERATURE REVIEW

According to Sarwar et al. (2001), RSs consist of the application of statistical and knowledge discovery techniques to

^a Giorgio Alfredo Spedicato is a Data Science and Actuarial Manager at the Unipol Group in Italy. Before joining the Unipol Group, he worked as P&C pricing and reserving actuary for the Italian subsidiaries of the Aviva and Axa Groups. He started his career as a freelance statistician.

He holds a Ph.D in Actuarial Science, and is a Fellow of the Casualty Actuarial Society (FCAS), the Society of Actuaries (FSA), and of the Italian Actuarial Professional Body (ONA). He is also a Certified Specialist in Predictive Analytics (CSPA) and Chartered Statistician (C.Stat).

^b Giuseppe Savino is Data Scientist and Solution Developer at the Zanichelli Group in Italy. He previously worked as a Lead Data Scientist for several consulting firms. He holds a B.S. in Physics and a M.S. in Applied Mathematics. His research interests focus on mathematical modeling of complex and chaotic systems.

the problem of making product recommendations based on previously recorded data. The objective of making such recommendations is the development of a value-added relationship between the seller and the consumer, by promoting cross-selling and conversion rates.

RSs date back to the early '90s with the Apriori algorithm (Agrawal and Srikant 1994). Basically, this technique returns a set of “if-then” rules that suggests the next purchase given the history of previous transactions (“market basket”) using simple probability rules. Nowadays, RSs are grouped into two macro categories: those using “collaborative filtering” (CF) reasoning and those using “content-based filtering” (CBF) (Ansari, Essegiaier, and Kohli 2000). Whereas the CF approach computes its predictions on the behavior of similar customers (Breese, Heckerman, and Kadie 1998), the CBF approach uses descriptions of both items and clients (or prospects) to understand the kinds of items a person likes (Aggarwal 2016). For an exhaustive discussion, Ricci et al. (2011) provide a broad overview.

As one may note from the increasing relevance of the RecSys conference (Association for Computing Machinery 2018), literature regarding RSs is vast and growing. Nevertheless, it is a topic little explored in actuarial and insurance-related research. In Mitra, Chaudhari, and Patwardhan (2014), we found an analysis of the potential of RSs in the II, but mostly from a business perspective and without an empirical approach. On the contrary, Rokach et al. (2013) underscore the difficulty of applying RSs in the II, addressing the issue of the small number of items to recommend. Nevertheless, the paper showed that adopting a simple CF algorithm to recommend riders could significantly benefit sales department productivity.

Qazi proposes the use of Bayesian networks to predict upselling and downselling (Qazi et al. 2017); however, the paper provides scarce details of the methodology.

It is important to stress that none of the techniques discussed in Mitra, Chaudhari, and Patwardhan (2014), Rokach et al. (2013), and Qazi et al. (2017) belongs to any model class with which actuaries would be familiar. For that reason, we offer an overview of the classical RS algorithms in the following sections.

3. THE FRAMEWORK

In this section we introduce how to build and evaluate a model based on machine learning algorithms, focusing our attention on the development of a recommendation system for insurance marketing. In particular, we stress that our final goal is the construction of an RS that can predict a client’s propensity to buy an insurance coverage in order to support business decisions.

3.1. METHODOLOGY

3.1.1. MODELING RATINGS DATA

Most classical RSs base their suggestions on a ratings matrix R that encodes information about customers and products. The name “ratings matrix” comes from the fact that originally each row contained customer ratings for sold products. In practice, the element r_{ij} could be expressed

in either a numeric-ordinal scale (for example, one-to-five-star rating) or using 0–1 data, sometimes referred to as “implicit rating” (IR). In other words, IR represents data gathered from user behavior where no rating has been assigned.

IR is the data format most relevant for our application: a value of 1 means that the policyholder has expressed a preference for (i.e., purchased) that specific insurance coverage, while a zero value can mean that the customer does not like the product, does not need it now, or is unaware of it. Thus, while the 1 class contains only positive samples, the zero class is a mixture of both positive and negative samples. In fact, it is not known whether the policyholder dislikes the item or would appreciate it, if he or she were aware of it. For the sake of completeness, other forms of IRs can be represented by the number of times the user purchased that specific product or by the times that product has been used (in the entertainment industry, for example).

A frequently used metric to compute product similarities in the implicit data context is the Jaccard index (Jaccard 1901), $J = \frac{|A \cap B|}{|A \cup B|}$, with $|A \cap B|$ representing the number of occurrences where products A and B are jointly held.

ML predictive algorithms are usually identified by a family of models. Such algorithms typically depend on the choice of one or more hyperparameters, within specific ranges, whose optimal value cannot usually be known a priori. The process of determining the best set of hyperparameters, known as “model tuning,” requires the testing of the algorithm over different choices of hyperparameters. This procedure is a fundamental step for the success of most ML models.

The simplest approach to this problem consists of the adoption of a Cartesian grid to sample over the hyperparameter space. Because this issue is crucial and deserves much thought, we address the interested reader to the work of Claesen and Moor (2015) for more refined techniques.

3.1.2. ASSESSING PERFORMANCE

An important consideration is the assessment of model performance. In particular, we need to understand how good a developed model is and how to compare alternative models.

Typically, the performance of a model is evaluated on a holdout data set, e.g., splitting customer data (usually the data set rows) into training and testing groups, or using a more computationally intensive k -fold cross-validation approach, which is a standard in predictive modeling (Kuhn and Johnson 2013).

When dealing with RS data, Hahsler (2018) suggests the following approach for the test group recommendations: withhold one or more purchases from their profiles, apply the chosen model, and assess how well the predicted items match those of the withheld ones. For binary and IR data, this reduces to verifying that the withheld items are contained within the top- N predicted items.

When ratings are produced for implicit data, binary classification accuracy metrics (like accuracy, AUC, log loss, and so forth) can be used to score and rank RS algorithms. In particular, the log loss metric measures the discrepancy between estimated $\hat{y}$ and actual probabilities of a target i -th event occurrence, y_i :

$$\text{logloss} = -\frac{1}{n} \sum_{i=1}^n [y_i \cdot \log_e(\hat{y}_i) + (1 - y_i) \cdot \log_e(1 - \hat{y}_i)].$$

On the other hand, the area under the receiver operating characteristic (ROC) curve (known as the AUC) measures the model's ability to classify the target (binary) outcomes. In fact, a model's output can be used in a decision-making process, defining a certain threshold and classifying cases as belonging either to the purchaser category or not whereby their score is higher than the set threshold. It is possible to cross-tabulate actual and predicted groups using something known as the "confusion matrix," which can be synthesized as follows, where $N = TP + FP + FN + TN$ is the total scored sample size:

Predicted	Observed: Event	Observed: Non-Event
Event	True Positive (TP)	False Positive (FP)
Non-Event	False Negative (FN)	True Negative (TN)

Performance metrics such as the following can be created on such a matrix:

- Accuracy, $\frac{TP+TN}{N}$
- Sensitivity (recall), $\frac{TP}{TP+FN}$
- Precision (positive predictive value), $\frac{TP}{TP+FP}$
- Specificity (true negative rate), $\frac{TN}{TN+FP}$
- Negative predictive value, $\frac{TN}{TN+FN}$

As Kuhn and Johnson (2013, 262-65) well detail, these metrics depend on the selected threshold value; in particular, there is a trade-off between sensitivity and specificity, and the AUC represents the area under all possible combinations of sensitivity and specificity as a function of the selected threshold.

Briefly, the log loss metric is used when it is necessary to estimate the probability as precisely as possible, while the AUC score is more appropriate when it is more important to distinguish between two classes (such as purchasers and non-purchasers for marketing strategy).

Furthermore, if two periods of data are available, a possible strategy is to build the RS based on the initial-period data in the training set and to assess how well the algorithm predicts purchases in the next period in the test set. This work adopts this assessment approach. We remark that the scoring is clearly performed on a product basis and is evaluated only on customers that do not hold that precise product in the initial period.

We stress that although splitting the data set into the training set and the test set is not necessary for all the approaches presented in this paper, for the sake of comparison of the algorithms, it has been shared across all the techniques.

We decided to use the ROC AUC as the metric to test the algorithms because of its ability as a ranking performance measure (Flach and Kull 2015; Hand and Till 2001). Our recommendation approach is not based on a classification task (i.e., client A is going to buy a specific product or not) but rather on a ranking task (i.e., client A is more inclined to buy a specific product than client B).

For that reason, we prefer the ROC AUC, although our data set presented highly unbalanced classes, as usually happens for some kinds of insurance coverages. The appen-

dix briefly addresses this specific issue.

3.2. THE AVAILABLE DATA AND MODELING LIMITATIONS

The chosen algorithms will be trained on an anonymized insurance marketing data set containing about 1 million policyholders' purchased coverages in two subsequent periods, as well as some demographic covariates. The examined approach's ability to predict next-period purchases (from the set of insurance coverages still not held) is measured. Supplementary binary variables indicate whether the policyholder purchased the coverage in period one and whether the policyholder purchased the coverage in period two; we use this supplementary information to assess the performance of the analyzed RSs.

We exclude CBF algorithms from our analysis, because the number of possible kinds of insurance coverage is lower by an order of magnitude than the number of items typically traded in online industries (e.g., Netflix, Amazon, Spotify, etc.). In fact, our application deals with a small number of products—the different kinds of insurance coverages—that show relative heterogeneity of characteristics. For example, a life or health policy is quite different from auto coverage. For this reason, if one client has both kinds of policies, while another one has just a life policy, the implication that the latter might also like an auto policy does not necessarily hold as easily as it might in other industries where the variety of products is much greater.

In addition, only two periods' worth of portfolio composition, rather than the full history of previous purchases, is available. For that reason, sequence-based algorithms, which have been found to be extremely useful in the literature, are not considered in this study.

3.3. MATHEMATICAL FORMULATION

With regard to the issue of recommendations in the insurance field, we present a mathematical framework capable of describing the functional setting between customers and products. We assume that the willingness of a person i to buy a product j may be represented by a probability density function P_j depending on the demographic characteristics of the person $\mathbf{d}_i$, the social pressure s_i , and the timing t_i .

It is reasonable to endow the aforementioned variables with the following characteristics:

- The demographic features (i.e., age, gender, wealth) identify the client's state and account for most of the P_j value.
- Strong social pressure might increase (or decrease) the likelihood of buying a product j —hence s_i can be regarded as a quantity that explains factors external to the client (i.e., advertisement campaigns or client networks).
- The timing is a random variable that models phenomena related to unpredictable events (e.g., a tennis player who loses his racket might need a new one)—thus, this variable is usually not detectable.

In an RS framework, typically product similarity bias is also often taken into account. However, as pointed out pre-

viously, in the insurance market the number of diverse products is small, and hence this bias effect, if present, can be neglected. For this reason, the initial assumption of a probability function P_j dependent only on a single product j rather than the entire set of products could be reasonable.

Notwithstanding this framework, finding the analytic formulation of the function P_j or discovering the dynamics of the problem is a complex task.

In the latter part of the following section, we propose a way of exploiting supervised learning algorithms for learning the function P_j explicitly based on the client's demographic features $\mathbf{d}_i$, disregarding the timing t_i and the social pressure s_i variables due to the lack of data. In particular, the next section is devoted to the implementation of all the classical RS approaches for the sake of completeness and comparison.

4. STATISTICAL TECHNIQUES

Herewith we provide an overview of different RS algorithms. In particular, we underscore that the power of these techniques lies in their generality. In fact, some of the algorithms presented have been and would be employed to tackle other problems of the actuarial sciences.

4.1. ARULES

Association rules (arules) is a well-established approach to finding interesting relationships in large databases (Agrawal and Srikant 1994). Following the notation of Hahsler, Grün, and Hornik (2005), let $I = \{i_1, i_2, \dots, i_n\}$ be a set of n binary attributes called items. Let $D = \{t_1, t_2, \dots, t_m\}$ be a set of transactions called the database. Each transaction in D has a unique transaction ID and contains a subset of the items in I . A rule is defined as an implication of the form $X \Rightarrow Y$ where $X, Y \subseteq I$ and $X \cap Y = \emptyset$. The sets of items (for short item sets) X and Y are called antecedent (left-hand-side or LHS) and consequent (right-hand-side or RHS) of the rule. Various measures of significance have been defined on rules, the most important of which are these:

- *support*, $\Pr(X \cap Y) = \text{supp}(X \cup Y)$, is defined as the proportion of transactions in the data set that contain that specific item set;
- *confidence* is the probability of finding the RHS given that the transaction contains the LHS:
 $\text{conf}(X, Y) = \Pr(Y|X)$; and
- *lift* is a measure of association strength, in terms of departure from independence:
 $\text{lift}(X, Y) = \frac{\Pr(X \cap Y)}{\Pr(X)\Pr(Y)}.$

The process of identifying rules can yield many of them, and thus we usually set a minimum constraint either on confidence or on support, or on both, in order to select “meaningful” rules. In addition, rules are often ranked according to the lift metrics to highlight more important associations. Nevertheless, the selection of the confidence and support thresholds is subjective given that no optimal metric was defined in the original arules model. The interested reader can consult Tan, Kumar, and Srivastava (2004) for a

deeper discussion of the topic.

The two most used algorithms to mine frequent item sets are arules and Eclat, with the latter the less well known; Borgelt (2003) provides the interested reader more details about their actual implementation. The arules algorithm has also been extended to work on supervised learning problems, as Hahsler (2018) discuss.

A great advantage of the arules approach is that it provides easily interpretable model output that can be easily communicated to nontechnical audiences, like senior management or producers, in the form of simple directives. A drawback of this technique is that there does not exist a direct measure of predictive performance, especially on one-class data.

4.2. USER-BASED AND ITEM-BASED COLLABORATIVE FILTERING

User-based collaborative filtering (UBCF) provides recommendations for a generic u_a user, identifying the k users most similar to u_a and averaging their ratings for the items not present in the u_a profile. It is assumed that because users with similar preferences would rate items similarly, missing ratings can be obtained by first finding a neighborhood of similar users and then aggregating their ratings to form a prediction (Hahsler 2018). A common measure of user similarity, at least for non-implicit data, is the cosine similarity (CS) coefficient,

$$s_{x,y} = \frac{\vec{x} * \vec{y}}{\|x\| * \|y\|},$$

which, clearly, is calculated only on the dimensions (items) that both users rated. For a given user a it is possible to define a neighborhood $N(a)$ selecting, for example, the k nearest neighbors by CS or another (dis)similarity metric. Then the estimated rating of the j th item for the a user is simply $\hat{r}_{aj} = \frac{1}{\sum_{i \in N(a)} s_{ai}} \sum_{i \in N(a)} s_{ai} r_{ij}$, where each i th neighbor rating r_{ij} has been weighted by its similarity with user a , s_{ai} . A serious drawback of UBCF is that it requires holding in memory the users' similarity matrix. This makes UBCF unfeasible on real data sets.

Another approach, item-based collaborative filtering (IBCF), calculates similarities within the items' distances set, whose dimensionality is usually orders of magnitude lower than the users' one. Then, given similarities between i th and j th items s_{ij} and the neighborhood $S(i)$ of the k items most similar to the i th, it is possible to estimate $\hat{r}_{ai} = \frac{1}{\sum_{j \in S(i) \cap \{k: r_{aj} \neq ?\}} s_{ij}} \sum_{j \in S(i) \cap \{k: r_{aj} \neq ?\}} s_{ij} r_{aj}$. As described further, this paper applies IBCF using the Jaccard index as a measure of products' distances. In fact, the CS coefficient is not suitable when only implicit data are available at the user level, as the jointly purchased products will just be flagged with ones. In general, the Jaccard index appears to be more frequently used when binary or implicit data are used.

Whereas UBCF requires a u^2 -sized similarity matrix to be kept in memory, IBCF requires an m^2 -sized one; an additional memory reduction is obtained by keeping in memory only the k most similar distances. Nevertheless, when the magnitude of the customers' size is relevant, only IBCF is used, and, empirically, it appears that moving from UBCF

to IBCF leads to only small performance losses according to Hahsler (2018).

4.3. MATRIX FACTORIZATION APPROACHES

Matrix factorization techniques, such as nonnegative matrix factorization and singular value decomposition (SVD), are the mathematical foundation of many RSs belonging to the CF family (Koren, Bell, and Volinsky 2009). The matrix factorization approach approximates the whole ranking matrix into a smaller representation:

$$R_{m \times n} \approx X_{m \times k} * Y_{k \times n}.$$

This artifact both reduces the problem's dimensionality and increases the computational efficiency. X has the same number of rows as the R matrix, but typically a much smaller number of columns, $k \ll n$. Y is a $k \times n$ matrix that represents the archetypal features set generated by combining columns from R , while X represents the projection of original users on the Y space. The result is twofold: first, each product is mapped on a reduced space representing its key dimension (archetypes); second, it is possible to compute products-dimension preferences for each given customer. For example, if we think of products as a combination of a few "essential" tastes, the Y matrix represents all items projected in the taste dimension, while the X matrix presents the projection of all users in the taste dimension. Therefore, the rating given by subject i to product j is estimated by $\hat{x}_{ij} = \tilde{x}_i * \tilde{y}_j^T$. Our analysis considers four algorithms in this category that represent different variations of the matrix decomposition approach — LIBFM, GLRM, ALS, and SLIM.

4.3.1. LIBFM

A typical solution for X and Y can be found solving

$$\min_{X,Y} \sum_{(u,v) \in R} \left[f(x_u, y_v; r_{u,v}) + \mu_X \|x_u\|_1 + \mu_Y \|y_v\|_1 + \frac{\lambda_X}{2} \|x_u\|_2^2 + \frac{\lambda_Y}{2} \|y_v\|_2^2 \right]$$

as shown in Chin et al. (2015), where f is the loss function and μ_P, μ_Q, λ_P , and λ_Q are penalty coefficients similar to the ridge and lasso ones of the elastic net regression model (Zou and Hastie 2005). For binary matrices, the logarithmic loss is typically used as well as binary matrix factorization (Z. Zhang et al. 2007). This algorithm is efficiently implemented in the LIBFM open-source library, from which it takes its name (Chin et al. 2016).

4.3.2. GLRM

Udell et al. (2016) recently introduced the generalized low rank model (GLRM) approach, which is another approach to effectively perform matrix factorization and dimensionality reduction. The compelling feature of this approach is that it can handle mixed-type data matrices, which means that the GLRM can handle matrices filled not only with numerical data but also with some columns of categorical, ordinal, binary, or count data. This is achieved by imposing not just a single loss function but many according to the category of the columns. This enables the GLRM to encompass and gen-

eralize many well-known matrix decomposition methods, such as PCA, robust PCA, binary PCA, SVD, nonnegative matrix factorization, and so on. As for LIBFM, the GLRM approach supports regularization, thus allowing the estimation process to encourage some structure such as sparsity and nonnegativity in X and Y matrices. Notable applications for GLRMs lie in missing values imputation, features embedding, and dimensionality reduction using latent variables.

4.3.3. ALS

The Alternating Least Squares (ALS) algorithm (Hu, Volinsky, and Koren 2008) was specifically developed for IR data. Used by Facebook, Spotify, and others, it has gained increasing popularity. That Apache Spark, a well-known open-source big-data environment, chose to implement ALS as the reference recommendation algorithm (Meng et al. 2016) for its machine learning engine (Mllib) confirms its importance. Like the previously mentioned ML methods, the ALS algorithm aims to decompose the rating matrix, $R_{m \times n} \approx U \times V$. U and V are initially randomly populated, and then least squares minimization is used in an iterative manner to obtain their optimal values. "Alternating" means that each loss minimization iteration comprises two steps: in the first, V is fixed and U is optimized, and in the second, the reverse happens. ALS is specifically suited for IR, since it introduces the concepts of manifest "preference" p_{ui} and confidence c , which is a measure of how explicit ratings reflect a true preference. Formulaically,

$$p_{ui} = \begin{cases} 1 & r_{ui} > 0 \\ 0 & r_{ui} = 0 \end{cases},$$

and, for confidence c_{ui} ,

$$c_{ui} = 1 + \alpha * r_{ui}.$$

This means that a minimal confidence exists even if the user has never purchased the product and that increasing interactions between the user and the product linearly increase the confidence. Hu, Volinsky, and Koren (2008) suggest values between 15 and 40 for α in their application. Clearly, when only single-purchase data (as in our insurance application) are available, confidence is of limited use, but it could be more useful in cases where several periods' worth of renewal history is available.

The loss function to be minimized becomes

$$\min_{x^*, y^*} \sum_{u,i} c_{ui} (p_{ui} - x_u^T y_i)^2 + \lambda \left(\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2 \right),$$

from which, after some algebra, the following alternating equations can be found:

$$\begin{cases} x_u = (Y^T Y + Y^T * (C^U - I) Y + \lambda I)^{-1} Y^T C^u p(u) \\ y_i = (X^T X + X^T * (C^i - I) X + \lambda I)^{-1} X^T C^i p(i) \end{cases},$$

where I is the identity matrix, X and Y are initially randomly filled, and, as for many of the other methods here-with presented, λ represents a penalty constant (usually set between 0.1 and 0.01). The similarity between item i and other items can be found by the following matrix operation: $V \times V_i^T$; as well, recommendations for user u and each item are derived by $U_i \times V^T$.

4.3.4. SLIM

The sparse linear method (SLIM) algorithm, as presented in Ning and Karypis (2011), blends ideas from matrix factorization and the robust linear model (as the elastic net regression). The recommendation score for the non-purchased item t_j for user u_i is given by $\tilde{a}_{ij} = a_i^T w_j$, where $a_{ij} = 0$ and w_j is a sparse-size column vector of aggregation coefficients. Thus, the SLIM model recommendation scores can be written in the following factorization form:

$$\tilde{A} = AW,$$

where A is the binary user-item purchase matrix and W is the $m \times m$ matrix of aggregation coefficients. In particular, the j column of W stands for w_j , and $\tilde{a}_i$ and a_i represent the row vectors for the recommendation and purchase histories of user i , respectively.

The W matrix solves the following regularized minimization optimization problem:

$$\begin{aligned} \min_W \quad & \frac{1}{2} \|A - AW\|_F^2 + \frac{\beta}{2} \|W\|_F^2 + \lambda \|W\|_1 \\ & W \geq 0 \\ & \text{diag}(W) = 0 \end{aligned},$$

where $\|\cdot\|_F$ is the Frobenius norm, and β and λ are the regularization parameters for an elastic net regression, respectively. The nonnegative constraint is applied on W such that the learned W matrix represents positive relations between items, if any. Imposing a null diagonal for W avoids trivial solutions (e.g., the identification of a matrix where an item always recommends itself) and ensures that a_{ij} is not used to compute $\tilde{a}_{ij}$.

It turns out that the columns of the W matrix are independent and this makes the computations easily parallelizable. Therefore, the SLIM algorithm efficiently scales on high-dimensional problems. The aforementioned paper shows an empirical analysis applying different CF algorithms on data sets frequently used to benchmark RSs, indicating that the SLIM algorithm often outperforms competing algorithms.

4.4. SUPERVISED LEARNING ALGORITHMS

Supervised learning (SL) techniques combine statistical inferential reasoning and approximation theory to estimate an unknown functional relation F between two spaces $\mathbf{X}$ and $\mathbf{Y}$ (Hastie, Tibshirani, and Friedman 2009), respectively denoted as the features space and the predictions space, once provided a set of sample points, called the training set.

$$\mathcal{S}_{\text{train}} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N \subset \mathbf{X} \times \mathbf{Y}$$

Canonically, the values of the elements in $\mathbf{X}$ are named predictors and the ones in $\mathbf{Y}$ are called labels. These algorithms, when properly configured, are capable of extracting patterns and correlations, constituted by a mapping

$$\hat{f} : \mathbf{X} \rightarrow \mathbf{Y} \quad | \quad \hat{f} \approx F$$

which generalizes over previously unseen samples, the test-set:

$$\mathcal{S}_{\text{test}} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^M \subset \mathbf{X} \times \mathbf{Y} \quad | \quad \mathcal{S}_{\text{test}} \cap \mathcal{S}_{\text{train}} = \emptyset$$

meaning that

$$\hat{f}(\mathbf{x}) \approx \mathbf{y} \quad \forall (\mathbf{x}, \mathbf{y}) \in \mathcal{S}_{\text{test}}.$$

The function $\hat{f}$ is typically found by solving the following constrained optimization problem through a gradient de-

scent method:

$$\begin{aligned} \underset{f}{\operatorname{argmin}} \quad & E_{\mathbf{y}}[l(\mathbf{y}, f(\mathbf{x})) | \mathbf{x}], \quad (\mathbf{x}, \mathbf{y}) \in \mathcal{S}_{\text{train}} \\ & E_{\mathbf{y}}[l(\mathbf{y}, f(\mathbf{x})) | \mathbf{x}] \approx E_{\mathbf{y}}[l(\mathbf{y}, f(\mathbf{x})) | \mathbf{x}], \\ & (\mathbf{x}, \mathbf{y}) \in \mathcal{S}_{\text{train}} \wedge (\mathbf{x}, \mathbf{y}) \in \mathcal{S}_{\text{test}}. \end{aligned}$$

In brief, supervised learning algorithms allow the construction of statistically solid forecasting models based on a given set of observations and associated outcomes. Linear regression is the simplest example of a supervised learning algorithm. But many other approaches familiar to actuaries—such as generalized linear models, or GLMs (McCullagh and Nelder 1989)—belong to the supervised learning class of ML algorithms.

These techniques are increasingly becoming a convenient and essential tool in the actuarial sciences: see, for example, Yang, Qian, and Zou (2018) for the adoption of ML in pure premium estimation; Spedicato, Dutang, and Petrini (2018) for an application in retention and conversion modeling; and the monographs of Frees, Derrig, and Meyers (2014, 2016) for a general overview. In our application, $\mathbf{x}$ may represent the customer's demographic profile and $\mathbf{y}$ the probability of a product's purchase. A key advantage of an SL approach compared with a CF approach is that it can be fit without any need of the history of previous purchases. Thus, it can provide recommendations not only to existing customers but also to prospective ones. Even if it is possible to use both demographic variables and the history of previous purchases in the SL approach, we choose not to do that in this application.

In what follows we give an overview of some of the more relevant supervised learning algorithms: gradient boosting trees (GBTs) and deep neural networks (DNNs). Although these ML models have usually been regarded as *black boxes*, research has been devoted to efforts to ease results explainability. In particular, we stress that algorithms exist capable of approximately measuring the marginal effect of single variables over the model's output and of investigating the overall variable importance. The interested reader is referred to Lundberg and Lee's (2017) seminal paper and to the DALEX and LIME libraries (Biecek 2018; Pedersen and Benesty 2018).

4.4.1. GENERALIZED LINEAR MODELS

Generalized linear models, or GLMs, represent a class of SL algorithms whose key assumptions are as follows: the i th observation of the dependent variable y_i belongs to the exponential family with mean and dispersion parameters μ_i and ϕ , respectively; the expected value and variance of y_i are, respectively, $E[y_i] = \mu_i = f(\eta_i)$ and $\text{Var}[y_i] = g(\mu_i) * \phi$; $f(\eta_i) = h(x_i)$, where h is a linear (or at least a linear combination of nonlinear terms) and f a monotone transformation, which is called "link function". Some link functions relevant for actuarial science applications are the logarithmic and the logit (i.e., $\log(\frac{x}{1-x})$).

Compared to many more recent ML algorithms, GLMs are easily estimated with most open source and corporate software tools (using the maximum likelihood approach). Another important feature is their inherent linearity, which strongly helps to explain modeling results. On the other hand, GLMs do not allow the full modeling of intrinsically

nonlinear relationships and they have no mechanism for discarding irrelevant predictors (feature selection) or for self-identifying significant interactions between predictors. Nevertheless, it is possible to fit GLMs using the elastic net approach instead of classical maximum likelihood, which helps to discard non-significant predictors and to make coefficients' estimates more stable. In addition, common approaches practitioners use to overcome nonlinearity are to transform continuous predictors into categorical ones (e.g., by binning into equal exposure groups, or decile binning) or to use nonlinear additive terms, such as polynomials or splines.

GLMs are the predictive models on which modern insurance pricing is founded, as is detailed in Goldburd, Khare, and Tevet (2016). For 20 years, loglinear Poisson and gamma models have been applied to model claims frequency and severity, while logistic regression has been used to estimate the retention and conversion probability of current and prospective policyholders as a function of rating factors. And a logistic regression would be a proper choice from within the GLM family to model product possession or propensity to purchase insurance coverages.

4.4.2. GRADIENT BOOSTING TREES

Gradient boosting tree (GBT) algorithms are based on an approximation technique, called boosting, that assumes that the function F can be approximated as a sum of a finite set of functions b_i , $F \approx \sum_{i=0}^N b_i$, being the b_i named "weak-learners":

$$b_i \equiv b(\mathbf{p}_i) : \mathbf{X} \rightarrow \mathbf{Y} \quad \forall i = 1, \dots, N$$

depending on a set of parameters $\mathbf{p}_i \in \mathbb{R}^P$. With these assumptions, the optimization problem introduced at the beginning of this section is transformed into a search for the optimal set of parameters $\{\mathbf{p}_i\}_{i=1}^P$ characterizing the boosters and minimizing the expected loss function over the training set, with the aforementioned generalization constraints.

From a computational point of view, the minimum is obtained via a gradient descent method constrained to the booster function b_i at each iteration i , the so-called *stage-wise* approach (Friedman 2000). In other words, at each step i of the optimization procedure, the optimal set of parameters $\mathbf{p}_i$ approximating $F - \sum_{j=1}^{i-1} b_j$ is sampled from the parameters space determining the weak learner b_i . In GBT algorithms, the boosters b are represented by shallow regression trees and, therefore, the parameters $\mathbf{p}$ are given by the ones characterizing the tree itself, such as the tree's depth or the number of leaves.

Regarding this issue, we decided to test three different implementations of GBT algorithms—XGBoost (Chen and Guestrin 2016), lightGBM (Ke et al. 2017), and CatBoost (Prokhorenkova et al. 2018)—in order to compare their performances. The name XGBoost stands for eXtreme Gradient BOOSTing. It was the first GBT implementation to significantly improve the time of performance, making possible the adoption of GBTs in a wide range of complex and data-demanding learning tasks. LightGBM improves the time of performance of XGBoost by adopting a novel numerical method for tree splitting, while CatBoost handles categorical features based on a smart encoding of the levels of cate-

gorical variables into numerical mappings.

4.4.3. DEEP NEURAL NETWORKS

DNN-based models embed the features space into the labels space, throughout the chained application of a sequence of smooth functions, called layers. Each layer l is typically constructed by a nonlinear function a , called an activation function, applied onto a linear combination of the layer inputs $\mathbf{x}_l$:

$$l : \mathbf{x}_l \rightarrow a(\mathbf{w}_l \mathbf{x}_l + \mathbf{b}_l),$$

where $\mathbf{w}_l$ are called the weights of the layer or neurons, and $\mathbf{b}_l$ refers to bias. We refer to the way multiple layers are combined to constitute the NN as network architecture $\mathcal{A}$.

The parameters $\{(\mathbf{w}_l, \mathbf{b}_l)\}_{l \in \mathcal{A}}$, characterizing the NN, are obtained by solving the supervised learning optimization problem. More specifically, the so-called *back propagation* procedure is applied to determine the network parameters minimizing the loss function. A more exhaustive introduction to DNNs can be found in Goodfellow, Bengio, and Courville (2016) and a review of DNNs applied to RSs can be found in S. Zhang et al. (2019).

In the last decade, thanks in part to progress made in the construction of CPUs and GPUs, NNs have been exploited in most ML problems due to their ability to spot complex patterns or highly nonlinear correlations by approximating complicated functions. In fact, to some extent, NNs can be regarded as very efficient nonlinear regressor models (Sarle 1994).

Despite the extreme accuracy of their performance, a major drawback of DNN is their "black-box" nature. In fact, it is difficult to explain why the given input properties produce the obtained results.

5. EMPIRICAL APPLICATION

We applied the previously reviewed techniques to a data set consisting of 1 million subjects who were customers for two consecutive years of a European insurance company. We considered the following group of variables for the analysis:

- Active holding of coverage for nine different categories of products: HAS_A, HAS_B, ..., HAS_I. Coverage possession was measured both in time zero and one.
- Fifteen categorical and 14 numerical features representing the policyholder's demographic profile measured in the first year only
- The policyholder unique ID
- A supplementary group of variables, BOUGHT_A, BOUGHT_B, ..., BOUGHT_E, indicating whether the policyholder acquired the product between the first and second year. For example, the conditions to create BOUGHT_E are $(HAS_{E_1} = 0) \wedge (HAS_{E_2} = 1)$. This set of variables was used as a target outcome to evaluate the predictive performance of the algorithms.

After preprocessing, the data set was split into a training set, accounting for 70% of the data, and a testing set, comprising the remaining 30%. The testing set was used only

Table 1. Insurance product possession, period one

HAS_A	HAS_B	HAS_C	HAS_D	HAS_E	HAS_F	HAS_G	HAS_H	HAS_I	HAS_L
0.78	0.22	0.01	0.21	0.02	0.02	0.02	0.01	0.02	0.03

for the evaluation of the results, and not during the training phase, to avoid any leakage of information. As Kuhn and Johnson (2013) explain, such a procedure is necessary for the proper construction and the consistent evaluation of any predictive model.

Data preprocessing and descriptive statistics were performed using the R and Python programming languages (R Core Team 2017; van Rossum 1995) as well as packages dedicated to implement the specific algorithms, which are specified in the following paragraphs. We also used other libraries for data management, such as the tidyverse (Wickham 2017) package. While in this section we describe the general implementation approaches for each algorithm herewith presented, the reader may find the full computational details in the code hosted in the [GitHub](#) repository. In addition, the appendix contains a few random rows of the anonymized data set used in the empirical part of the paper.

5.1. PRODUCT POSSESSION DISTRIBUTION

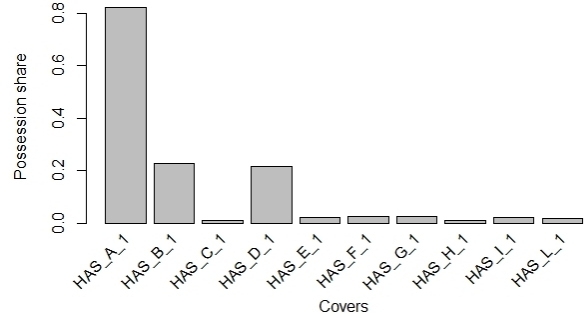
[Table 1](#) represents product possession in the initial period.

[Table 2](#) shows the forthcoming-period purchases, on which the recommender systems' outputs will be assessed. It can be seen that the purchase of an additional policy, for existing customers, is an infrequent event. Depending on the product, it varies from a minimum value of 0.1% to a maximum value of 2.1%. We take into account the low frequency of the target event as we develop the models.

5.2. ARULES

We used the R implementation of the Apriori algorithm in the arules library (Hahsler, Grün, and Hornik 2005). As detailed in the source code, the rule mining requires transforming the transaction data set into a sparse matrix format. In order to identify rules, we must set minimum confidence and support in advance. The increase of the values of both parameters restricts the number of candidate rules, reducing the number of retrieved ones. In addition, one must also set the minimum length of rules found when setting the algorithm. As previously discussed, the choice of such values is mostly subjective and depends on the analyst's preference; because no goodness-of-fit performance is defined, there is no need for hyperparameter optimization in this context. For our analysis, we judgmentally set the rules' search with the following criteria:

- Minimum rule length: 2
- Minimum support: 1%—that is, both LHS and RHS must exist in the data set in at least 1% of row transactions
- Minimum confidence: 20%, which means that the probability of RHS occurrence, given LHS, should be at least 20%

**Figure 1. Covers active in the initial period****Table 2. Purchases within the periods**

	x
BOUGHT_A	0.0117
BOUGHT_B	0.0212
BOUGHT_C	0.0011
BOUGHT_D	0.0155
BOUGHT_E	0.0018
BOUGHT_F	0.0019
BOUGHT_G	0.0037
BOUGHT_H	0.0017
BOUGHT_I	0.0018
BOUGHT_L	0.0038

Only two rules were found. [Table 3](#) shows them, sorted by decreasing lift.

If we add the constraint of positive association (lift greater than one), we are left with only one rule. Basically, that rule says to suggest purchasing B to all customers that have already purchased coverages A and D since this holds for 38% of customers holding products A and D. We tested this rule on policyholders that do not have the B coverage at time zero, and we measured the AUC of predicting actual B purchased in the 0–1 period using the simple rule identified by arules. The AUC is just slightly higher than the noninformative value of 0.5, 0.52 precisely, while the two other binary classification metrics, the true positive rate (also known as sensitivity) and the true negative rate (specificity), are, respectively, 0.12 and 0.92. The low sensitivity indicates a poor ability in detecting customers that are willing to purchase the product.

Table 3. arules association rules found

Rules	Support	Confidence	Lift	Count
{HAS_A_1,HAS_D_1} => {HAS_B_1}	0.038598	0.3784303	1.659620	22050
{HAS_B_1,HAS_D_1} => {HAS_A_1}	0.038598	0.8186679	1.000111	22050

Table 4. Item–item Jaccard similarity matrix

	A	B	C	D	E	F	G	H	I	L
A	0.00	0.22	0.00	0.11	0.00	0.00	0.00	0	0.00	0.00
B	0.22	0.00	0.00	0.12	0.00	0.00	0.00	0	0.00	0.00
C	0.00	0.00	0.00	0.00	0.00	0.00	0.04	0	0.00	0.04
D	0.11	0.12	0.00	0.00	0.00	0.00	0.00	0	0.00	0.00
E	0.00	0.00	0.00	0.00	0.00	0.08	0.00	0	0.10	0.00
F	0.00	0.00	0.00	0.00	0.08	0.00	0.00	0	0.04	0.00
G	0.00	0.00	0.04	0.00	0.00	0.04	0.00	0	0.00	0.00
H	0.00	0.00	0.00	0.00	0.06	0.00	0.00	0	0.04	0.00
I	0.00	0.00	0.00	0.00	0.10	0.04	0.00	0	0.00	0.00
L	0.00	0.00	0.04	0.04	0.00	0.00	0.00	0	0.00	0.00

5.3. ITEM-BASED COLLABORATIVE FILTERING

We discarded the user-based collaborative filtering (UBCF) approach because it requires the complete between-users distance matrix. This would make the model unfeasible for real business. We use the recommenderlab R package (Hahsler 2018) implementation of the item-based collaborative filtering (IBCF) approach to fit the model. This R package implements many CF algorithms as well as an infrastructure for evaluating RSs for both continuous and binary ratings. For each product, the Jaccard similarity index is calculated with all other distinct products, but only the top k are kept. The IBCF approach requires the k number to be specified in advance. The CS metric would not be appropriate for binary data, since it must be computed on commonly purchased pairs and the value on the rating matrix would always be one. Taking into account the relatively small number of alternative coverages, we subjectively set k equal to two. A few justifications for that choice follow. As with matrix factorization approaches, we wanted to significantly reduce the dimensionality of the problem, and thus the square root of the number of available products would be the upper bound for k : $k < \sqrt{m}$. In addition, using the two most similar products would make the algorithm easier to communicate to and be accepted by a nontechnical audience. The resulting Jaccard similarity weighting matrix J is shown in Table 4.

Matrix non-null entries $j_{i,j}$ represent the similarity score (the Jaccard index, J) between product i and j ; J values lower than the top k ones per row have been zeroed. J values are also used as a propensity-to-purchase score for product j for users holding product i . For each user, the products' propensity scores have been computed as shown

Table 5. IBCF suggestions: AUC by product

Product	ibcf_auc
A	0.53
B	0.51
C	0.57
D	0.52
E	0.69
F	0.66
G	0.53
H	0.61
I	0.56
L	0.55

in Hahsler (2018, 7). As was done when estimating the arules model, we computed the AUC for purchases of each product k for policyholders not having product k beginning at time 0 as shown in Table 5.

5.4. MATRIX FACTORIZATION APPROACHES

In general, based on our judgment, we chose that the dimension of the archetypal space would be between two and three, while the number of columns of the rating matrix—that is, the number of different insurance coverages—would be 10. The following paragraphs set forth the models' specific implementation details.

5.4.1. LIBFM

We used the R implementation of LIBFM from the recosys-system package (Qiu et al. 2017). The package requires that the data be put in long format; thus there is one observation for each rating made by a policyholder. Also, it requires one to specify the actual loss function to minimize (in our case, the choice was obviously log loss) and some tuning parameters: the dimensionality of the matrix factorization in terms of number of latent factors (k), ridge and lasso parameters. As shown below, we can perform a grid search to identify the optimal choice of the tuning parameters:

1. k matrix dimension: 2 and 3
2. loss: (log)loss
3. learning rate: 0.05 and 1
4. costp_l1, costq_l1: 0.05 and 0
5. costp_l2, costq_l2: 0.001 and 0

After retrieval of the optimal configuration of tuning parameters, the optimal parameters can be used to retrieve the final model. Nevertheless, the overall performance of the model is quite poor, as [Table 6](#) shows.

5.4.2. GLRM

We used the R implementation of the generalized low rank model (GLRM) algorithm provided by the H2O platform (LeDell et al. 2018). The H2O software currently supports the top ML suites. H2O efficiently implements most relevant supervised (random forest, deep learning, GLM, GBM, etc.) and unsupervised (K-means, PCA, GLRM) algorithms supporting parallelization, GPU training, and an easy structure to perform hyperparameter optimization and scoring. Fitting the GLRM requires that the input rank matrix be provided to the H2O functions and also that the following (major) parameters be specified:

- the loss functions by columns;
- the k rank of the X and Y matrices;
- the type of regularization (ridge, “L1,” or lasso, “L2”) chosen; and
- the regularization weights on X and Y .

As for most ML models, the optimal hyperparameter configuration cannot be retrieved analytically and must be found using a grid search approach. A generalized loss (known as the “final objective value”) is defined for the GLRM, and it is possible to identify parameters’ configuration within a grid that minimizes such a search (using, e.g., a random grid search), similar to the LIBFM approach. The H2O documentation suggests to randomly set missing a percentage of the input matrix entries (e.g., 20%) and to evaluate the ability of the matrix decomposition solution to “reconstruct” the complete matrix (as measured by the final objective value).

After having found the best configuration of parameters, we apply that configuration on our GLRM in order to find the X and Y columns. The reconstructed matrix $X * Y$ is then used as scores to identify purchase propensity for each product (by columns). The AUC of GLRM scores on time-one purchases is shown in [Table 7](#).

The AUCs are higher than 0.5 (thus some predictive per-

Table 6. LIBMF suggestions: AUC by product

Product	libfm_auc
A	0.50
B	0.50
C	0.47
D	0.50
E	0.52
F	0.51
G	0.52
H	0.51
I	0.50
L	0.51

Table 7. GLRM suggestions: AUC by product

Product	glrm_auc
A	0.53
B	0.51
C	0.70
D	0.56
E	0.73
F	0.64
G	0.60
H	0.75
I	0.63
L	0.59

formance is evident) and in some cases are fairly high (> 0.70).

5.4.3. ALS

To the best of our knowledge, the only efficient implementation of the ALS algorithm without setting up a Spark cluster is the *implicit* Python package (Frederickson 2018). That library provides an efficient implementation of various implicit rating matrices factorization algorithms. A three-factor solution was fit, leaving the α and λ parameters as default.

The actual predictive performance on the test set was nevertheless negligible, as [Table 8](#) shows.

5.4.4. SLIM

We used the R implementation of the SLIM algorithm in the slimrec package (Srikanth 2017). The estimation procedure is as follows:

1. Find the optimal weight between the ridge and lasso regression, α .
2. Estimate W given the best alpha on the training set.
3. Estimate the recommendation scores,

$$\tilde{A}_{\text{test}} = A_{\text{test}} \times W.$$

Table 9 displays the predictive performance of the SLIM model. As the table displays, for some products, it can be relatively high.

A section in the appendix shows the estimated W matrix and an example of the calculation of predicted ratings for a customer sample.

5.5. SUPERVISED LEARNING MODELS

5.5.1. OVERVIEW

Taking into consideration the available client data and the supervised learning settings, we trained for each policy j in our set of coverages the chosen GBT algorithms by providing them the demographic characteristics $\mathbf{d}_i$ as inputs $\mathbf{X}_i$, once properly preprocessed, and the possessions of a fixed product at time as labels $\mathbf{Y}_i^j$ (i.e., Y_i^j is 1 when the client i holds the coverage j and 0 otherwise). All the algorithms we employ for this analysis minimize the log loss during the training phase, since we want to model a probability. For the NN, we trained only one network for the sake of time constraints; therefore, we adopted the vectors of possession as labels and the categorical cross-entropy as minimizing loss.

The function f , learned through the prescribed training procedure, provides as output $\hat{Y}_i^j$ numbers between 0 and 1, is a close approximation of a probability (Kuhn and Johnson 2013), and represent a score of how much a client i is inclined to hold the coverage j , conditioned to the inputs $\mathbf{X}_i$: a numerical representation of the P_j functions introduced in the previous section given the available data.

Consistent with what has been done before, the models were constructed using first-period training data and using HAS_X (i.e., possession of product X) as a response variable. Then, predictions have been calculated for test set data. Finally, we calculate the AUC comparing the above predictions to the $BOUGHT_X$ variable (i.e., the purchase of product X between period one and two) on the test set records that have $HAS_X = 0$ in the first period.

Recalling our objectives, it is worth noting that when training the algorithms, we are assuming that the propensity of owning a product is shadowing the one of buying that same product. This fact is in contrast with how supervised learning algorithms have been presented in the previous section, the canonical theoretical framework. Here, instead, we are testing the derived models on a response that is different than the one on which they are trained, but related.

As previously shown, the actual period-one purchases are significantly lower than 1% for many products in our data set. Thus, while it was theoretically possible to model directly the one-year probability to purchase, estimated probabilities would be affected by a much higher uncertainty.

5.5.2. GENERALIZED LINEAR MODELS

We used logistic regressions, fitting separate models for possession of each product in the initial period. To handle

Table 8. GLRM suggestions: AUC by product

Product	als_auc
A	0.50
B	0.50
C	0.52
D	0.50
E	0.50
F	0.49
G	0.51
H	0.49
I	0.51
L	0.50

Table 9. SLIM suggestions: AUC by product

Product	slim_auc
A	0.54
B	0.56
C	0.69
D	0.57
E	0.75
F	0.71
G	0.62
H	0.75
I	0.64
L	0.59

potential nonlinearities, continuous predictors were transformed into categorical ones using tercile binning (i.e., each new variable has three possible levels that indicate in which tercile the original value lies). Coefficients were estimated using the elastic net approach, performing a grid search over the global penalty weight parameter α . The R version of the H2O software was used to fit the GLMs.

All of the AUC values are significantly above the non-informative threshold, and many products report an AUC above 0.70 indicating good classification ability, as Table 10 indicates.

5.5.3. MACHINE LEARNING MODELS: GBT AND DEEP LEARNING

We employed three boosting algorithms: XGBoost, lightGBM, and CatBoost. For all the GBT algorithms we pre-processed the categorical variables in order to group the states accounting for less than the first percentile in a new state, labeled “others.” In more detail, we performed a one-hot encoding on the categorical variables to train XGBoost and a label encoding for lightGBM. We did not need to apply any transformations for CatBoost because it accepts categorical features.

All of these algorithms have different sets of hyperparameters, which were selected through threefold cross-validation, trained over a representative subset of the data. Since the procedure is time-consuming, we tested only a subset of all the possible admissible parameters. Hence, we cannot assure the reader of having found the optimal hyperparameters for our data set, but this was not an issue in this work.

For the deep learning (DL) algorithm, the input data were preprocessed to obtain numerical inputs contained in the $[0, 1]$ interval, which is a requirement of the algorithm. On categorical variables, we applied an entity-embedding layer (Guo and Berkahn 2016), which encodes categorical features into a k -dimensional hypercube $C = [0, 1]^k$, where k is smaller than the number of the categorical classes. Meanwhile on numerical features, we used a min-max scaling transformation. We adopted Keras with a TensorFlow backend as a DL framework (Chollet 2017). To construct the presented network, we tested several architectures; however, as with the GBT algorithms, we cannot guarantee that the provided solution is the optimal one.

5.5.4. RESULTS

[Table 11](#) displays the AUC values for the supervised learning algorithms. As expected, the predictive performance is always significantly higher compared to the canonical RS approaches, and to the GLMs, even if to a lesser extent. Furthermore, we note that there is no absolute winner among the chosen techniques, since the differences among them are small and could be attributed to suboptimal hyperparameter choices.

6. SUMMARY AND CONCLUDING REMARKS

The scarce literature we have reviewed underscores the difficulty of using RSs in the II due to the cardinality of the product portfolio, which is almost always orders of magnitude smaller than in other industries. The small number of products offered might also have an impact on the performance and efficacy of some of the algorithms we discuss. Nevertheless, widespread interest in RSs has arisen in the ML community, and given the role RSs could play in supporting insurance underwriting, we believe actuaries should be aware of such systems' potential, even if that may be limited compared to their application in the e-commerce and entertainment industries.

We applied common RSs to a real II data set, and that allows us to compare the algorithms in different dimensions: predictive accuracy, easiness of implementation, and interpretability.

In terms of predictive accuracy, what is of most interest to the insurer is a model's ability to target existing policyholders' willingness to purchase additional coverage not currently in their portfolio. In this regard, the AUC on forthcoming-period purchases can be thought of as a reasonably accurate metric.

[Table 12](#) reports each RS model's AUC by product. We see that the ALS and LIBFM algorithms perform poorly on this data set, as their AUCs indicate little if any discriminating ability. On the other hand, the respective performances

Table 10. Logistic GLMs: AUC by product

Product	auc_glm
A	0.56
B	0.60
C	0.73
D	0.64
E	0.72
F	0.67
G	0.77
H	0.77
I	0.71
L	0.71

Table 11. ML models: AUC by product

Product	CTB	DL	LGB	XGB
A	0.66	0.66	0.66	0.64
B	0.61	0.60	0.61	0.59
C	0.73	0.73	0.75	0.72
D	0.65	0.65	0.65	0.64
E	0.75	0.73	0.75	0.74
F	0.70	0.68	0.71	0.69
G	0.79	0.76	0.79	0.79
H	0.77	0.76	0.79	0.78
I	0.73	0.75	0.76	0.73
L	0.72	0.71	0.73	0.72

Note: CTB = CatBoost; DL = Deep Learning ; LGB = lightGBM; XGB = XGBoost.

of the supervised learning methods are very similar and generally good. SLIM offers a good predictive performance, close to the ability of the other supervised learning models. The GLRM matrix decomposition technique displays some discriminating ability, but certainly lower than the above-mentioned models. Finally, the IBCF approach displays a fair discriminating ability, with values that are generally higher than the noninformative value of 0.5. The poor performance of the ALS and LIBFM algorithms could be because of their need for a much larger number of columns in order to work properly. It could be of some interest to confirm such findings using different and possibly wider-ranging insurance-marketing-related data sets.

Calculating the average rank of each RS model (shown in [Table 13](#)) provides a quantitative supplement for the analysis. Here, a lower average rank indicates superior predictive performance:

We have omitted the arules model from the analysis since it was not possible to provide rules for all products and since the provided rules offered a predictive performance only slightly above the noninformative value.

While the SL models offer superior performance, they

Table 12. RS models' AUC on next-period purchases

Product	IBCF	LIBFM	GLRM	ALS	SLIM	GLM	CTB	DL	LGB	XGB
A	0.53	0.50	0.53	0.50	0.54	0.56	0.66	0.66	0.66	0.64
B	0.51	0.50	0.51	0.50	0.56	0.60	0.61	0.60	0.61	0.59
C	0.57	0.47	0.70	0.52	0.69	0.73	0.73	0.73	0.75	0.72
D	0.52	0.50	0.56	0.50	0.57	0.64	0.65	0.65	0.65	0.64
E	0.69	0.52	0.73	0.50	0.75	0.72	0.75	0.73	0.75	0.74
F	0.66	0.51	0.64	0.49	0.71	0.67	0.70	0.68	0.71	0.69
G	0.53	0.52	0.60	0.51	0.62	0.77	0.79	0.76	0.79	0.79
H	0.61	0.51	0.75	0.49	0.75	0.77	0.77	0.76	0.79	0.78
I	0.56	0.50	0.63	0.51	0.64	0.71	0.73	0.75	0.76	0.73
L	0.55	0.51	0.59	0.50	0.59	0.71	0.72	0.71	0.73	0.72

do require much more modeling, data preparation efforts, and computational timing, orders of magnitude higher than what most CF models require. This is especially true for the ML models (deep neural networks and gradient boosting trees), which also almost always suffer from lack of interpretability. They require not only dependent variables (product possession) but also available demographic predictors to be preprocessed. Also many different models must be tested, and all of the presented models involved hyperparameter tuning. IBCF involves that also, but on the other hand, IBCF tuning is generally the fastest among the presented models. It appears that the SLIM model offers a very good balance between computational timing and predictive performance: its average rank score is just below that of the GLMs, without the need for the data preparation and computing time that GLMs require.

It is our impression that only the arules model, and to a lesser extent the IBCF approach, provides a directly intelligible output. In fact, association rules can be expressed in “if-then” rules form, and IBCF is based on an item–item similarity matrix that can somehow be intuitive. The GLMs can still be considered interpretable models, but they could require more effort than the aforementioned approaches. This consideration is valid also for the SLIM approach, where the product suggestions can be seen as a weighted combination of the existing product portfolio. The other models are either different forms of matrix decomposition or nonlinear or tree-based classification models (the SL models), and only recently have tools been developed to improve the methods' interpretability.

Herein we have explored only a fraction of the methodologies used in the burgeoning field of RSs as at the end of 2018. A broad and practical presentation of many relevant RS algorithms implemented in modern Python software can be found in Javier Rodríguez Zaurin's GitHub repository (Zaurin 2019).

Table 13. Average rank of the RS models

Model	Rank
LGB	1.6
CTB	2.5
XGB	3.6
EE	3.7
GLM	4.7
SLIM	5.2
GLRM	6.9
IBCF	7.8
LIBFM	9.4
ALS	9.6

ACKNOWLEDGMENTS

The work was sponsored by the Casualty Actuarial Society (CAS), the Actuarial Foundation's Research Committee, and the Committee on Knowledge Extension Research of the Society of Actuaries (SOA) during 2018. The authors wish to thank Jody Allen, professional education and research coordinator (CAS), and Erika Schultzy, research associate (SOA), for their support.

The authors would also like to thank Marco Bocchio for constructive criticism of the manuscript.

We especially thank the anonymous reviewers for their careful reading of our manuscript and their many insightful comments and suggestions, which have significantly improved this work.

Finally, the opinions expressed in the paper are solely those of the authors. Their employers guarantee neither the accuracy nor the reliability of the content provided herein and take no position on it.

Submitted: December 28, 2018 EST, Accepted: February 15, 2020 EST

REFERENCES

- Aggarwal, Charu C. 2016. *Recommender Systems: The Textbook*. Springer International.
- Agrawal, Rakesh, and Ramakrishnan Srikant. 1994. "Fast Algorithms for Mining Association Rules." In *VLDB '94: Proceedings of the 20th International Conference on Very Large Data Bases*, edited by Jorge B. Bocca, Matthias Jarke, and Carlo Zaniolo, 487–99. San Francisco: Morgan Kaufmann.
- Ansari, Asim, Skander Essegaier, and Rajeev Kohli. 2000. "Internet Recommendation Systems." *Journal of Marketing Research* 33 (3): 363–75. <https://doi.org/10.1509/jmkr.37.3.363.18779>.
- Association for Computing Machinery. 2018. *RecSys '18: Proceedings of the 12th ACM Conference on Recommender Systems*. New York: ACM.
- Biecek, Przemyslaw. 2018. "DALEX: Explainers for Complex Predictive Models." <https://arxiv.org/abs/1806.08915>.
- Borgelt, Christian. 2003. "Efficient Implementations of Apriori and Eclat." In *FIMI '03: Proceedings of the IEEE ICDM Workshop on Frequent Itemset Mining Implementations*, edited by Bart Goethals and Mohammed J. Zaki.
- Breese, John S., David Heckerman, and Carl Kadie. 1998. "Empirical Analysis of Predictive Algorithms for Collaborative Filtering." In *UAI '98: Proceedings of the 14th Conference on Uncertainty in Artificial Intelligence*, edited by Gregory F. Cooper and Serafin Moral, 43–52. San Francisco: Morgan Kaufmann.
- Chen, Tianqi, and Carlos Guestrin. 2016. "XGBoost: A Scalable Tree Boosting System." <http://arxiv.org/abs/1603.02754>.
- Chin, Wei-Sheng, Bo-Wen Yuan, Meng-Yuan Yang, Yong Zhuang, Yu-Chin Juan, and Chih-Jen Lin. 2016. "LIBMF: A Library for Parallel Matrix Factorization in Shared-Memory Systems." *Journal of Machine Learning Research* 17 (1): 2971–75.
- Chin, Wei-Sheng, Yong Zhuang, Yu-Chin Juan, and Chih-Jen Lin. 2015. "A Fast Parallel Stochastic Gradient Method for Matrix Factorization in Shared Memory Systems." *ACM Transactions on Intelligent Systems and Technology* 6 (1): 2–24. <https://doi.org/10.1145/2668133>.
- Chollet, François. 2017. *Deep Learning with Python*. Manning Publication.
- Claesen, Marc, and Bart De Moor. 2015. "Hyperparameter Search in Machine Learning." ArXiv:1502.02127.
- Flach, Peter. 2019. "Performance Evaluation in Machine Learning: The Good, the Bad, the Ugly, and the Way Forward." Association for the Advancement of Artificial Intelligence. <https://doi.org/10.1609/aaai.v33i01.33019808>.
- Flach, Peter, and Meelis Kull. 2015. "Precision-Recall-Gain Curves: PR Analysis Done Right." In *Advances in Neural Information Processing Systems 28*, edited by C. Cortes, N.D. Lawrence, D.D. Lee, M. Sugiyama, and R. Garnett, 838–46. Curran Associates, Inc. <http://papers.nips.cc/paper/5867-precision-recall-gain-curves-pr-analysis-done-right.pdf>.
- Frederickson, Ben. 2018. "SciPy: Fast Python Collaborative Filtering for Implicit Datasets in Python." <https://implicit.readthedocs.io/en/latest/quickstart.html>.
- Frees, Edward W., Richard A. Derrig, and Glenn Meyers, eds. 2014. *Predictive Modeling Applications in Actuarial Science*. Vol. 1, Predictive Modeling Techniques. International Series on Actuarial Science. New York: Cambridge University Press.
- , eds. 2016. *Predictive Modeling Applications in Actuarial Science*. Vol. 2, Case Studies in Insurance. International Series on Actuarial Science. New York: Cambridge University Press. <https://doi.org/10.1017/cbo9781139342681>.
- Friedman, Jerome H. 2000. "Greedy Function Approximation: A Gradient Boosting Machine." *Annals of Statistics* 29: 1189–1232.
- Goldburd, Mark, Anand Khare, and Dan Tevet. 2016. "Generalized Linear Models for Insurance Rating." Arlington, VA: Casualty Actuarial Society.
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. Adaptive Computation and Machine Learning Series. Cambridge, MA: MIT Press.
- Guo, Cheng, and Felix Berkhahn. 2016. "Entity Embeddings of Categorical Variables." <http://arxiv.org/abs/1604.06737>.
- Hahsler, Michael. 2018. *Recommenderlab: Lab for Developing and Testing Recommender Algorithms*. <http://CRAN.R-project.org/package=recommenderlab>.
- Hahsler, Michael, Bettina Grün, and Kurt Hornik. 2005. "arules—A Computational Environment for Mining Association Rules and Frequent Item Sets." *Journal of Statistical Software* 14 (15): 1–25. <https://doi.org/10.18637/jss.v014.i15>.
- Hand, David J., and Robert J. Till. 2001. "A Simple Generalisation of the Area under the ROC Curve for Multiple Class Classification Problems." *Machine Learning* 45 (2): 171–86. <https://doi.org/10.1023/a:1010920819831>.
- Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York: Springer. <http://www-stat.stanford.edu/~tibs/ElemStatLearn/>.
- Hu, Yifan, Chris Volinsky, and Yehuda Koren. 2008. "Collaborative Filtering for Implicit Feedback Datasets." In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining*, 263–72. Washington, DC: IEEE Computer Society. <https://doi.org/10.1109/icdm.2008.22>.

- Jaccard, Paul. 1901. "Étude Comparative de La Distribution Florale Dans Une Portion Des Alpes Et Des Jura." *Bulletin de la Société vaudoise des sciences naturelles* 37: 547–79.
- Ke, Guolin, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. "LightGBM: A Highly Efficient Gradient Boosting Decision Tree." In *Advances in Neural Information Processing Systems 30*, edited by I. Guyon, U.V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, 3146–54. Curran Associates, Inc. <http://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree.pdf>.
- Koren, Yehuda, Robert Bell, and Chris Volinsky. 2009. "Matrix Factorization Techniques for Recommender Systems." *Computer* 42 (8): 30–37. <https://doi.org/10.1109/mc.2009.263>.
- Kuhn, Max, and Kjell Johnson. 2013. *Applied Predictive Modeling*. New York: Springer.
- LeDell, Erin, Navdeep Gill, Spencer Aiello, Anqi Fu, Arno Candel, Cliff Click, Tom Kraljevic, et al. 2018. *h2o: R Interface for the 'H2O' Scalable Machine Learning Platform*. <https://CRAN.R-project.org/package=h2o>.
- Lundberg, Scott M., and Su-In Lee. 2017. "A Unified Approach to Interpreting Model Predictions." In *Advances in Neural Information Processing Systems 30*, edited by I. Guyon, U.V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, 4765–74. Curran Associates, Inc. <http://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf>.
- McCullagh, P., and J.A. Nelder. 1989. *Generalized Linear Models*. 2nd ed. Boca Raton, FL: Chapman & Hall/CRC. <https://books.google.it/books?id=UzmDDwAAQBAJ>.
- Meng, Xiangrui, Joseph Bradley, Burak Yavuz, Evan Sparks, Shivaram Venkataraman, Davies Liu, Jeremy Freeman, et al. 2016. "Machine Learning in Apache Spark." *Journal of Machine Learning Research* 17 (34): 1–7.
- Mitra, Sanghamitra, Nilendra Chaudhari, and Bipin Patwardhan. 2014. "Leveraging Hybrid Recommendation System in Insurance Domain." *International Journal of Engineering and Computer Science* 3 (10).
- Netflix. 2009. "The Netflix Prize." *Proceedings of KDD Cup and Workshop 2007* (3/6): 3–6. <https://doi.org/10.1145/1562764.1562769>.
- Ning, Xia, and George Karypis. 2011. "SLIM: Sparse Linear Methods for Top-N Recommender Systems." In *ICDM '11: Proceedings of the 2011 11th IEEE International Conference on Data Mining*, 497–506. Washington, DC: IEEE Computer Society. <https://doi.org/10.1109/icdm.2011.134>.
- Pedersen, Thomas Lin, and Michaël Benesty. 2018. *lime: Local Interpretable Model-Agnostic Explanations*. <https://CRAN.R-project.org/package=lime>.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2018. "CatBoost: Unbiased Boosting with Categorical Features." In *Advances in Neural Information Processing Systems 31*, edited by S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, 6639–49. Curran Associates, Inc. <http://papers.nips.cc/paper/7898-catboost-unbiased-boosting-with-categorical-features.pdf>.
- Qazi, Maleeha, Glenn M. Fung, Katie J. Meissner, and Eduardo R. Fontes. 2017. "An Insurance Recommendation System Using Bayesian Networks." In *RecSys '17: Proceedings of the Eleventh ACM Conference on Recommender Systems*, 274–78. New York: Association for Computing Machinery. <https://doi.org/10.1145/3109859.3109907>.
- Qiu, Yixuan, Chih-Jen Lin, Yu-Chin Juan, Wei-Sheng Chin, Yong Zhuang, Bo-Wen Yuan, Meng-Yuan Yang, et al. 2017. *recoSystem: Recommender System Using Matrix Factorization*. <https://CRAN.R-project.org/package=recoSystem>.
- R Core Team. 2017. *R: A Language and Environment for Statistical Computing*. Vienna: R Foundation for Statistical Computing. <https://www.R-project.org/>.
- Ricci, Francesco, Lior Rokach, Bracha Shapira, and Paul B. Kantor. 2011. *Recommender Systems Handbook*. Boston, MA: Springer.
- Rokach, Lior, Guy Shani, Bracha Shapira, Eyal Chapnik, and Gali Siboni. 2013. "Recommending Insurance Riders." In *SAC '13: Proceedings of the 28th Annual ACM Symposium on Applied Computing*, 253–60. New York: Association for Computing Machinery. <https://doi.org/10.1145/2480362.2480417>.
- Rossum, Guido van. 1995. "Python Tutorial." Report CS-R9526. Amsterdam: Centrum voor Wiskunde en Informatica (CWI).
- Sarle, Warren S. 1994. "Neural Networks and Statistical Models." In *Proceedings of the Nineteenth Annual SAS Users Group International Conference*, 1538–50. Cary, NC: SAS Institute. https://people.orie.cornell.edu/dayidr/or474/nn_sas.pdf.
- Sarwar, Badrul, George Karypis, Joseph Konstan, and John Riedl. 2001. "Item-Based Collaborative Filtering Recommendation Algorithms." In *Proceedings of the 10th International Conference on World Wide Web*, 285–95. New York: Association for Computing Machinery. <https://doi.org/10.1145/371920.372071>.
- Spedicato, Giorgio Alfredo, Christophe Dutang, and Leonardo Petrini. 2018. "Machine Learning Methods to Perform Pricing Optimization. A Comparison with Standard GLMs." *Variance: Advancing the Science of Risk* 111 (2): 69–89.
- Srikanth, K.S. 2017. *slimrec: Sparse Linear Method to Predict Ratings and Top-N Recommendations*. <https://CRAN.R-project.org/package=slimrec>.
- Tan, Pang-Ning, Vipin Kumar, and Jaideep Srivastava. 2004. "Selecting the Right Objective Measure for Association Analysis." *Information Systems* 29 (4): 293–313. [https://doi.org/10.1016/s0306-4379\(03\)00072-3](https://doi.org/10.1016/s0306-4379(03)00072-3).

- Udell, Madeleine, Corinne Horn, Reza Zadeh, and Stephen Boyd. 2016. "Generalized Low Rank Models." *Foundations and Trends in Machine Learning* 9 (1): 1–118. <https://doi.org/10.1561/22000000055>.
- Wickham, Hadley. 2017. *tidyverse: Easily Install and Load the "Tidyverse"*. <https://CRAN.R-project.org/package=tidyverse>.
- Yang, Yi, Wei Qian, and Hui Zou. 2018. "Insurance Premium Prediction via Gradient Tree-Boosted Tweedie Compound Poisson Models." *Journal of Business & Economic Statistics* 36 (3): 456–70. <https://doi.org/10.1080/07350015.2016.1200981>.
- Zaurin, Javier Rodriguez. 2019. *RecoTour*. GitHub Repository. <https://github.com/jrzaaurin/RecoTour>.
- Zhang, Shuai, Lina Yao, Aixin Sun, and Yi Tay. 2019. "Deep Learning Based Recommender System: A Survey and New Perspectives." *ACM Computing Surveys* 52 (1): 5.
- Zhang, Zhongyuan, Tao Li, Chris Ding, and Xiangsun Zhang. 2007. "Binary Matrix Factorization with Applications." In *ICDM 2007: Proceedings of the Seventh IEEE International Conference on Data Mining*, 391–400. Los Alamitos, CA: IEEE Computer Society. <https://doi.org/10.1109/icdm.2007.99>.
- Zou, Hui, and Trevor Hastie. 2005. "Regularization and Variable Selection via the Elastic Net." *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 67 (2): 301–20. <https://doi.org/10.1111/j.1467-9868.2005.00503.x>.

APPENDIX

Table 14. Product possession in the first period

ID	A_1	B_1	C_1	D_1	E_1	F_1	G_1	H_1	I_1	J_1
263	1	0	0	0	0	0	0	0	0	0
464153	1	0	0	1	0	0	0	0	0	0
837508	1	0	0	0	0	0	0	0	0	0
582959	1	1	0	0	0	0	0	0	0	0
442859	1	0	0	0	0	0	0	0	0	0

Table 15. Product possession in the second period

A_1	B_1	C_1	D_1	E_1	F_1	G_1	H_1	I_1	J_1
1	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0

SAMPLE ANONYMIZED ROWS OF THE DATA SETS

PRODUCT POSSESSION

Tables 14 and 15 display product possession for a few sample customers in the first and second periods.

Categoricals and continuous predictors for the same customers follow.

SLIM DETAILS

The estimated coefficient matrix of the SLIM model $\widehat{W}$ is shown in Table 18.

Thus, recalling that given a (possibly implicit) rating matrix $R_{m \times n}$, and that the estimated SLIM ratings are given by $R * \widehat{W}$, we can estimate ratings for the sampled customers (Table 19).

SCORING

Recommender systems can be regarded as binary classifiers: will a client buy the recommended product or not? From a modeling perspective, that may be a legitimate point of view; however, since most of the ML binary classifiers produce scores between 0 and 1 rather than dichotomic outputs, it is more meaningful to treat them as ranking tools.

Furthermore, behind this idea there lies part of the mathematical framework introduced previously: the studied phenomenon has an intrinsic stochastic nature that cannot be captured via a simple binary response. When one

chooses a threshold for a binary classifier it fixes the capabilities of the model, collapsing the information represented by the scores.

Nonetheless, a threshold needs to be fixed at some point and the binary nature of the phenomenon has to be taken into account.

In particular, it is worth noting that the subtle objective of an RS in the binary classifier framework is maximizing the number of false negative samples, what we call the prospect clients. This fact implies that a model that performs well on holding data (in the test set in the first period) might be an arbitrarily bad RS once tested on the purchasing data (the test set in the second period).

For this reason we choose log loss minimization, rather than AUC maximization, for the training phase of the supervised learning algorithms: our intent is to capture the mathematical formulation described in Subsection 3.3, not the construction of a solid binary classifier. We are not aware of valid alternatives to this choice, and future research might be devoted to finding a loss function that could take into consideration also the false negative ratio argument.

Regarding the threshold, we must also consider that the proportion of clients buying a specific coverage is typically low for almost all of our data set coverages, and hence the classes are unbalanced. This issue makes the ROC AUC a less informative index on the quality of the models when compared under a classification perspective.

Table 16. Categorical covariates

CAT01	CAT02	CAT03	CAT04	CAT05	CAT06	CAT07	CAT08	CAT09	CAT10	CAT11	CAT12	CAT13	CAT14	CAT15
B	A	a_3	b_3	B	A	c_8	A	d_1	A	e_1	NA	g_3	h_7	i_19
B	A	a_2	b_0	A	A	c_2	A	d_0	A	e_0	NA	g_1	h_1	i_1
B	A	a_25	b_3	B	A	c_7	A	d_0	A	e_0	NA	g_2	h_3	i_9
A	A	a_10	b_3	B	A	c_0	A	d_6	A	e_5	NA	g_0	h_0	i_0
B	A	a_66	b_1	B	A	c_13	A	d_18	A	e_11	NA	g_2	h_6	i_17

Table 17. Numeric covariates

NM01	NM02	NM03	NM04	NM05	NM06	NM07	NM08	NM09	NM10	NM11	NM12	NM13	NM14
75	11	11	10	25	0	0	503	485	515	0	1450	489	934
67	2	0	38	40	0	0	514	490	514	0	600	496	956
85	2	9	9	22	0	0	507	470	521	0	850	492	958
46	2	17	23	56	0	0	516	470	522	0	1200	496	953
62	21	72	21	32	0	0	519	511	510	0	1600	502	940

Table 18. W matrix for SLIM model

	A	B	C	D	E	F	G	H	I	L
A	0.00	0.20	0.00	0.09	0.00	0.00	0.01	0.00	0.01	0.01
B	0.48	0.00	0.02	0.11	0.01	0.01	0.02	0.01	0.01	0.02
C	0.00	0.37	0.00	0.21	0.03	0.04	0.10	0.02	0.00	0.07
D	0.22	0.10	0.01	0.00	0.01	0.01	0.01	0.01	0.02	0.03
E	0.00	0.06	0.02	0.14	0.00	0.13	0.00	0.06	0.16	0.01
F	0.00	0.07	0.02	0.09	0.11	0.00	0.05	0.03	0.04	0.02
G	0.03	0.13	0.04	0.11	0.00	0.05	0.00	0.02	0.04	0.02
H	0.00	0.10	0.03	0.10	0.13	0.08	0.07	0.00	0.08	0.03
I	0.04	0.07	0.00	0.16	0.15	0.04	0.04	0.03	0.00	0.03
L	0.01	0.17	0.05	0.29	0.01	0.03	0.03	0.02	0.03	0.00

Table 19. SLIM model estimated ratings for products not already held

A	B	C	D	E	F	G	H	I	L
	0.2	0.00	0.09	0.00	0.00	0.01	0.00	0.01	0.01
	0.3	0.01		0.02	0.02	0.02	0.01	0.03	0.03
	0.2	0.00	0.09	0.00	0.00	0.01	0.00	0.01	0.01
		0.02	0.19	0.01	0.01	0.03	0.01	0.02	0.02
	0.2	0.00	0.09	0.00	0.00	0.01	0.00	0.01	0.01

Table 20. ML models: performance metrics

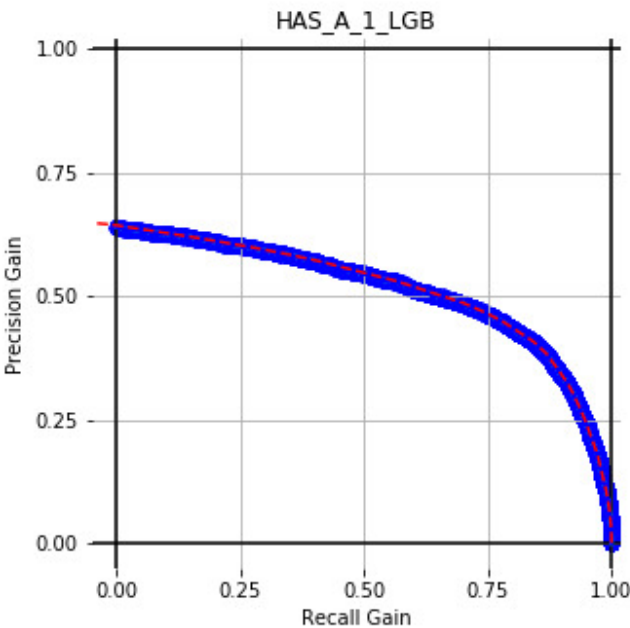
A	B	C	D	E	F	G	H	I	L
	0.2	0.00	0.09	0.00	0.00	0.01	0.00	0.01	0.01
	0.3	0.01		0.02	0.02	0.02	0.01	0.03	0.03
	0.2	0.00	0.09	0.00	0.00	0.01	0.00	0.01	0.01
		0.02	0.19	0.01	0.01	0.03	0.01	0.02	0.02
	0.2	0.00	0.09	0.00	0.00	0.01	0.00	0.01	0.01

Here, we must emphasize that the precision-recall AUC, sometimes proposed as a good score for these situations, is not a valid alternative (Flach and Kull 2015). Instead, the precision-recall-gain (PRG) AUC represents a possible meaningful indicator for assessing the quality of an unbalanced classifier. We refer the reader interested in how to score ML models in general to Flach’s (2019) paper “Performance Evaluation in Machine Learning: The Good, the Bad, the Ugly, and the Way Forward.”

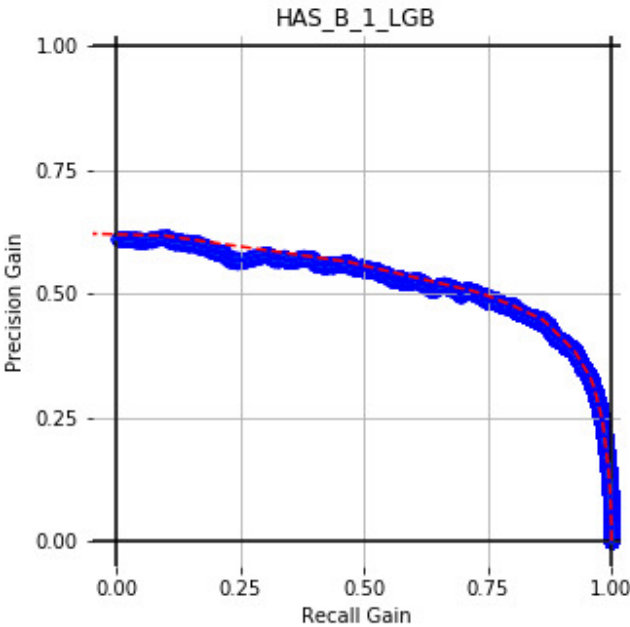
We evaluated the best SL models with respect also to the PRG AUC for the sake of completeness as [Table 20](#) shows, but further investigations are required. In the first instance, the LGB model seems to provide the most promising results.

The subsections that follow show AUC-ROC and PRG-AUC curves for all models for illustrative purpose.

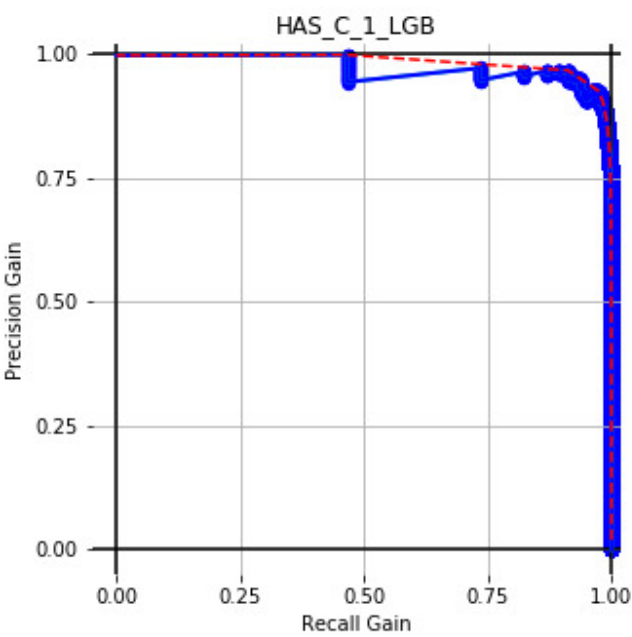
PRG-AUC curves



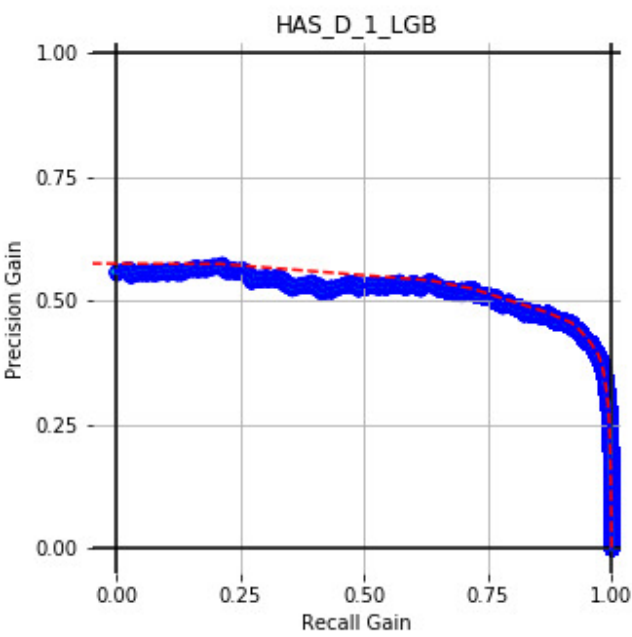
PRODUCT A PRG-AUC



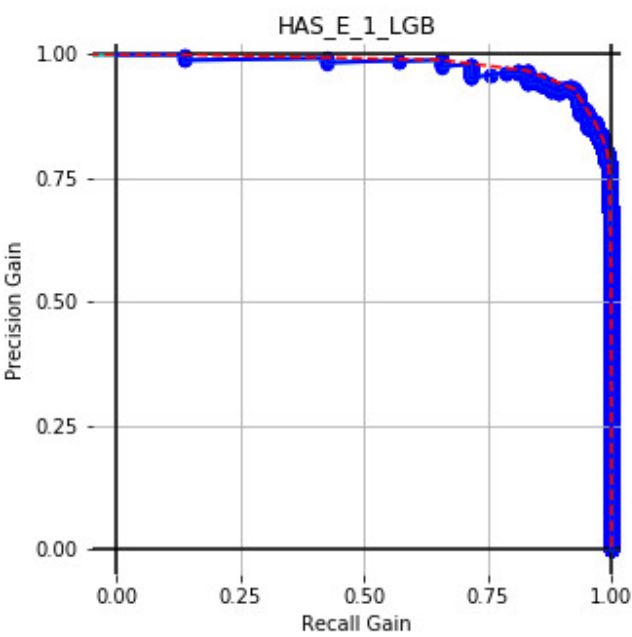
PRODUCT B PRG-AUC



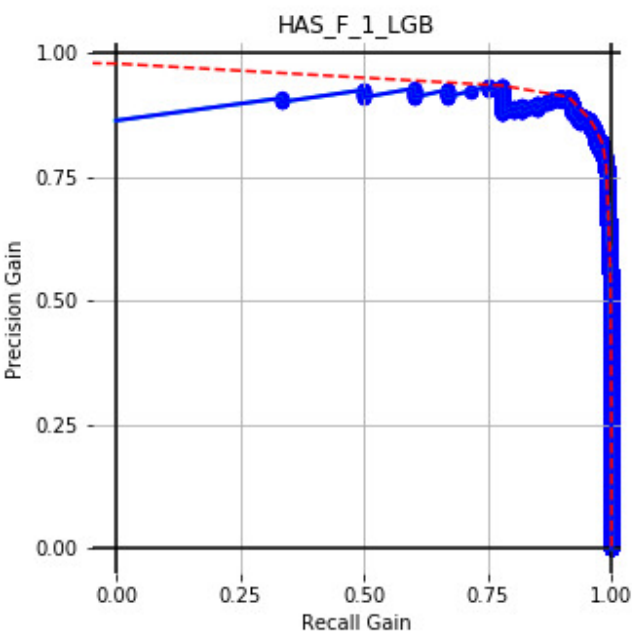
PRODUCT C PRG-AUC



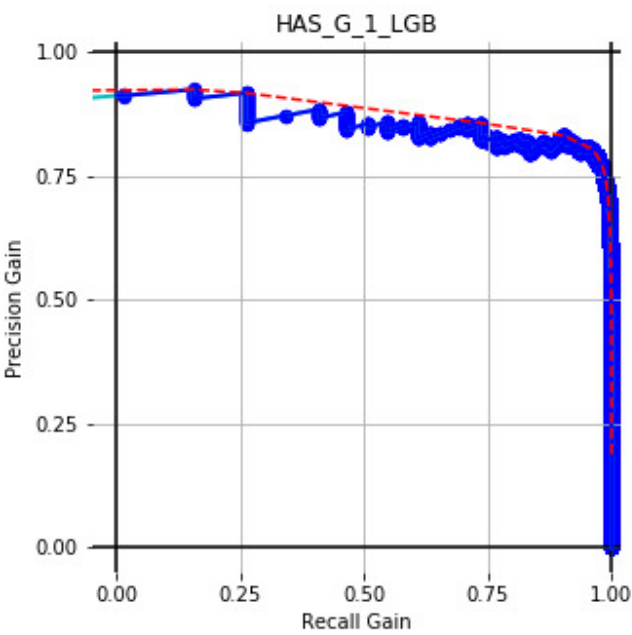
PRODUCT D PRG-AUC



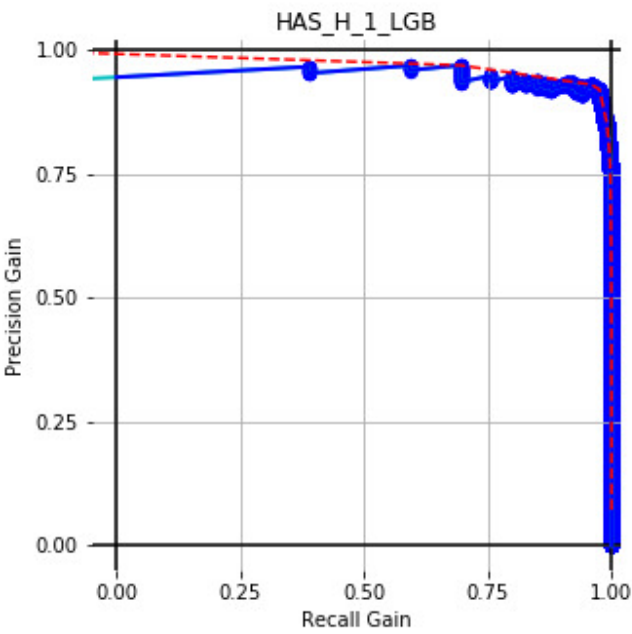
PRODUCT E PRG-AUC



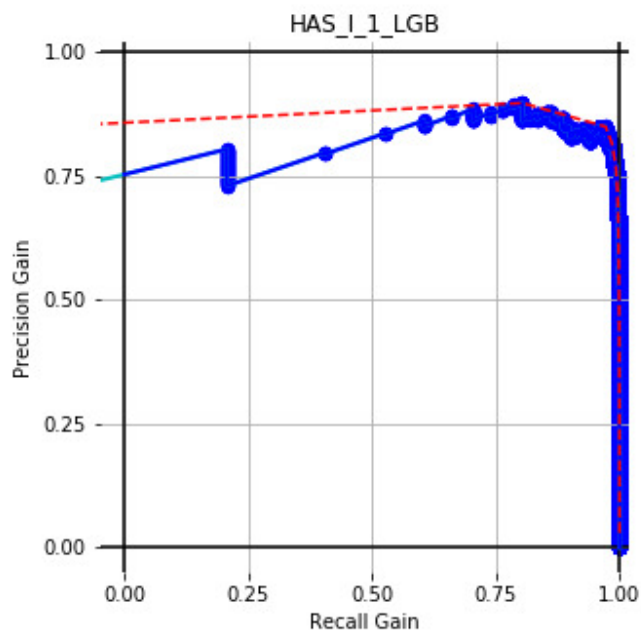
PRODUCT F PRG-AUC



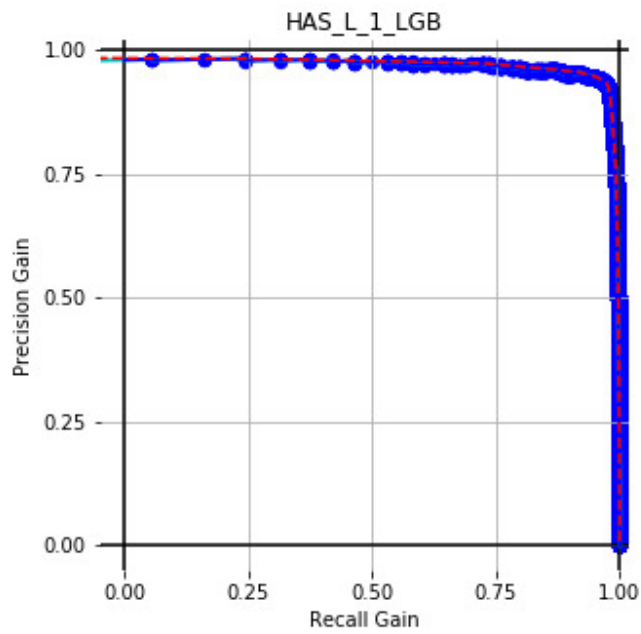
PRODUCT G PRG-AUC



PRODUCT H PRG-AUC

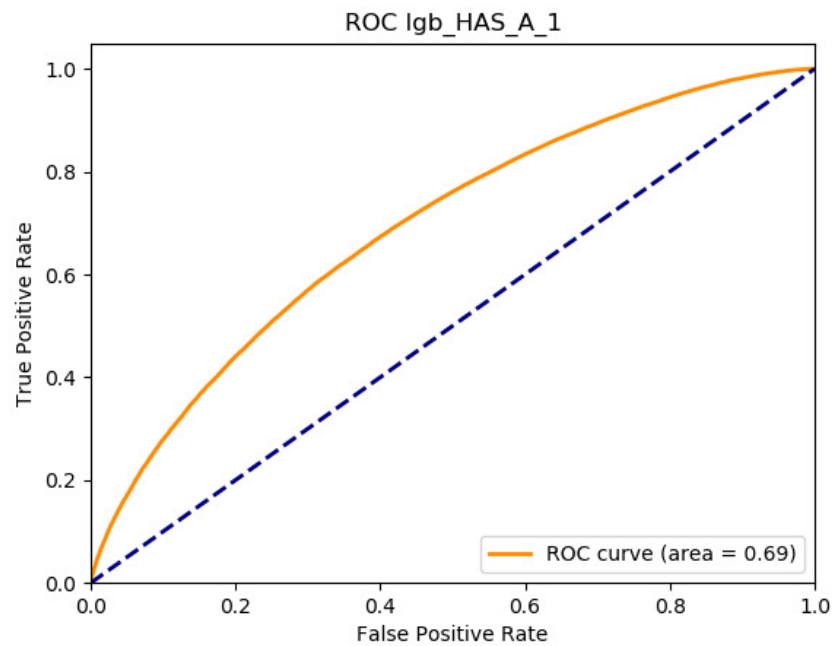


PRODUCT I PRG-AUC

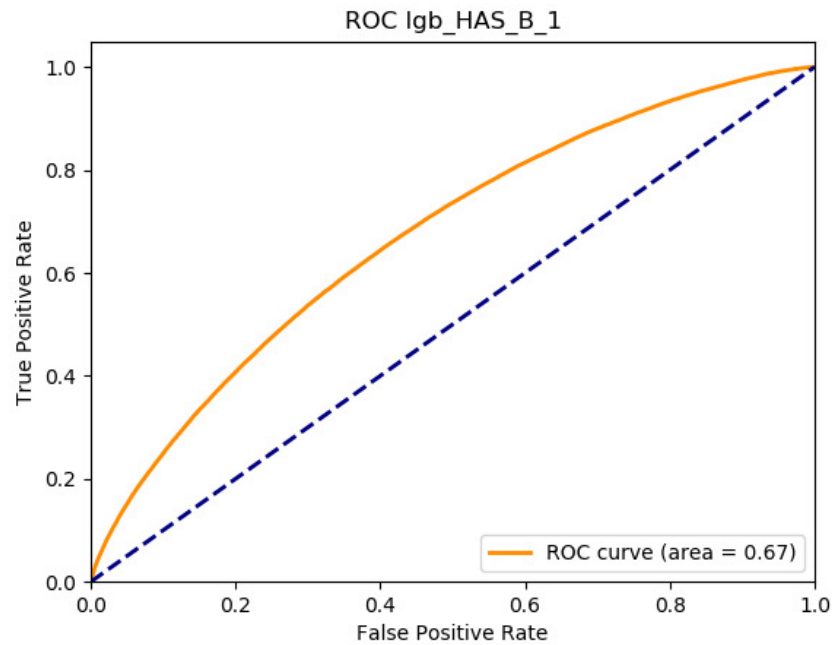


PRODUCT L PRG-AUC

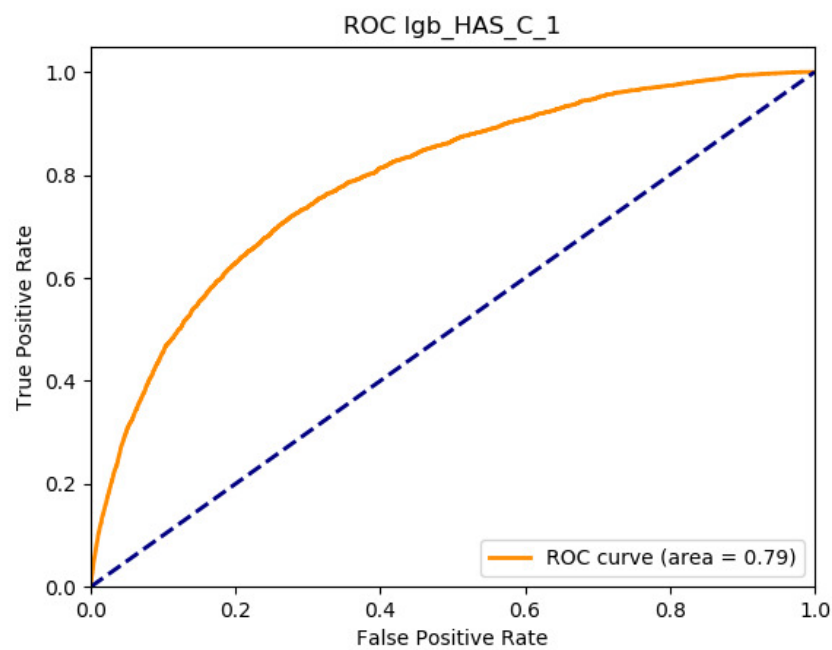
ROC-AUC curves



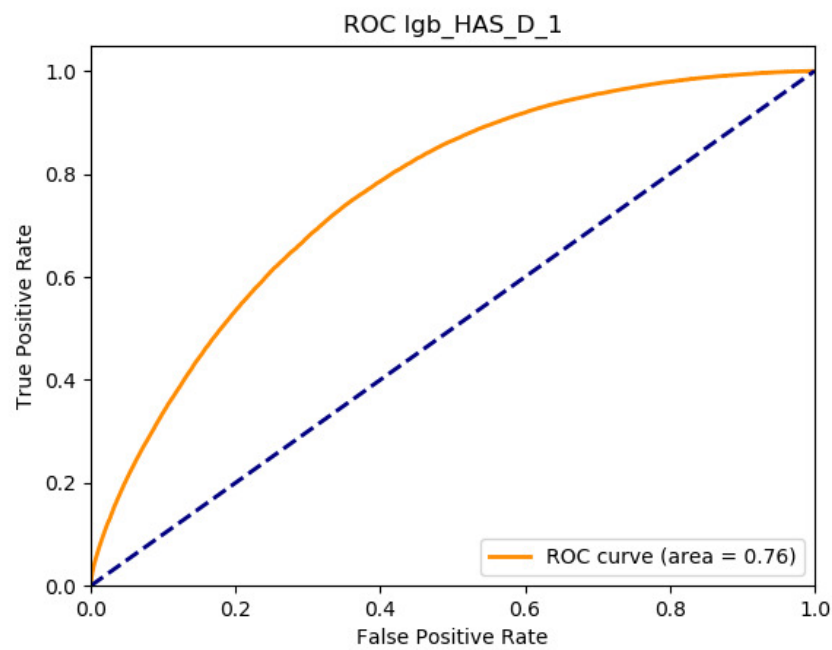
PRODUCT A ROC-AUC



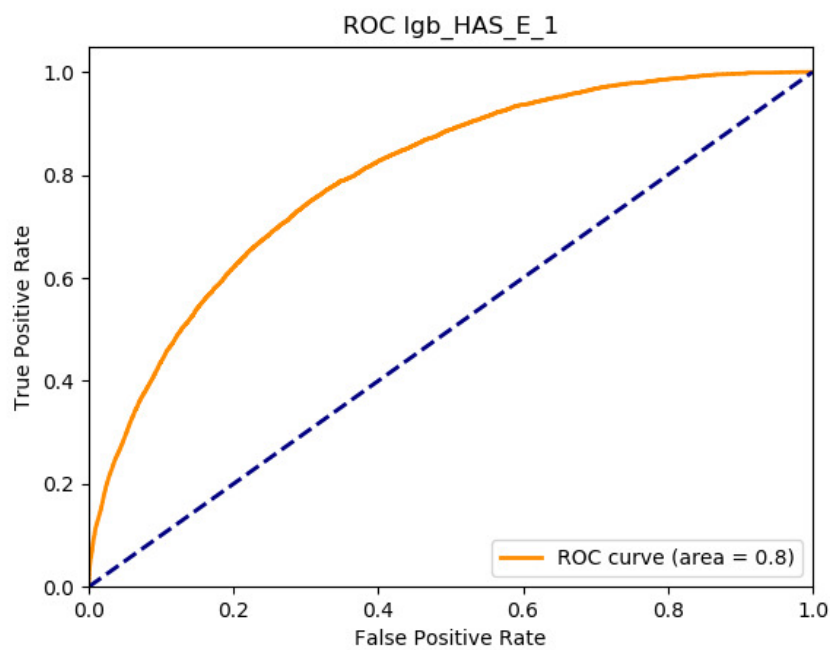
PRODUCT B ROC-AUC



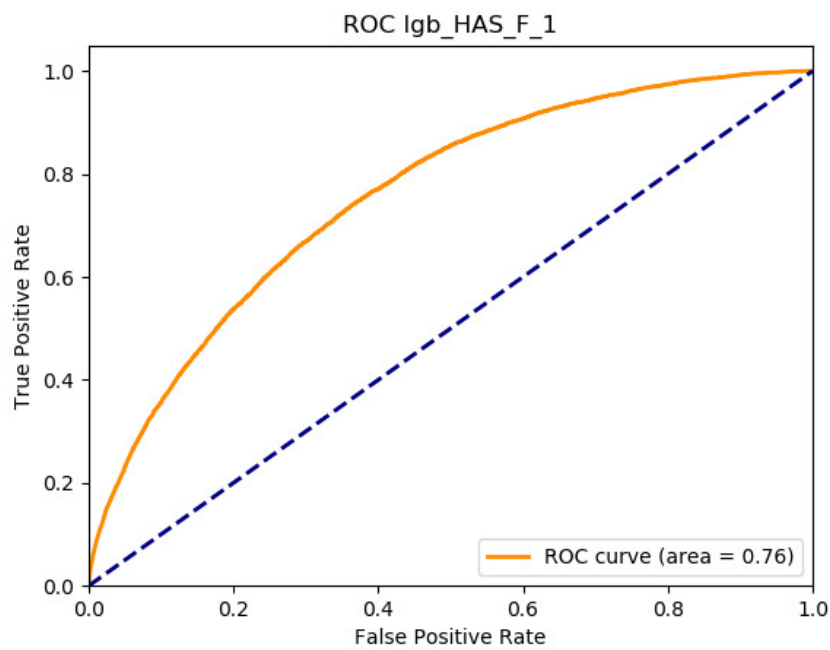
PRODUCT C ROC-AUC



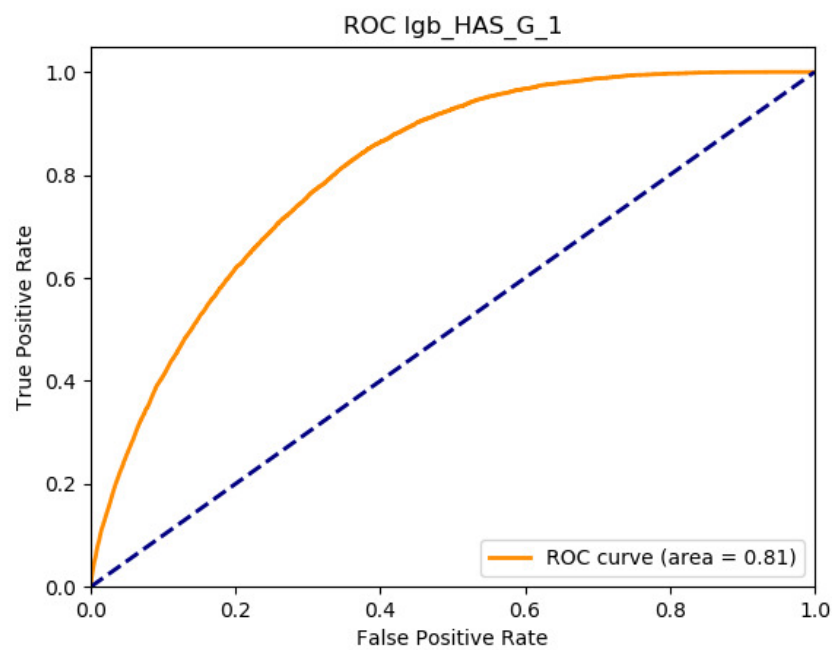
PRODUCT D ROC-AUC



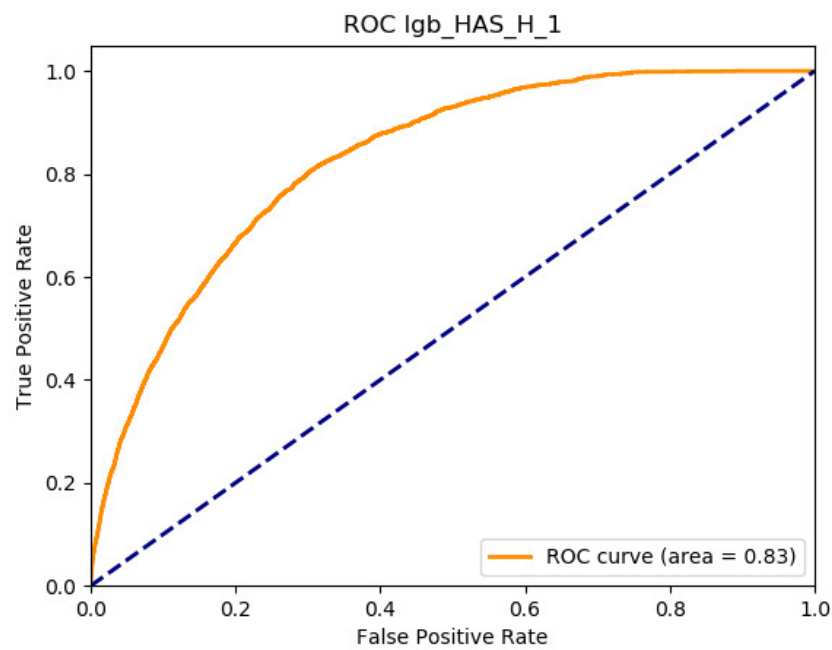
PRODUCT E ROC-AUC



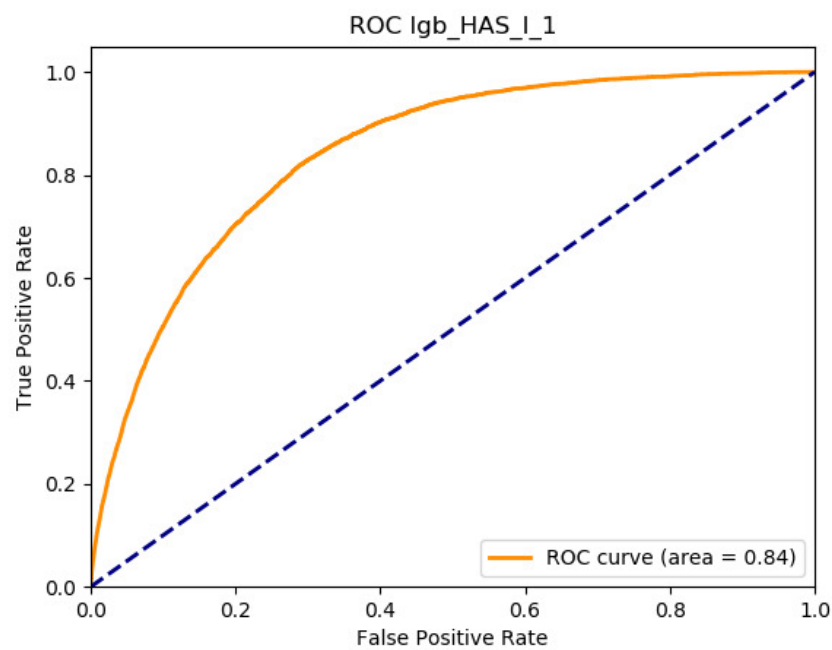
PRODUCT F ROC-AUC



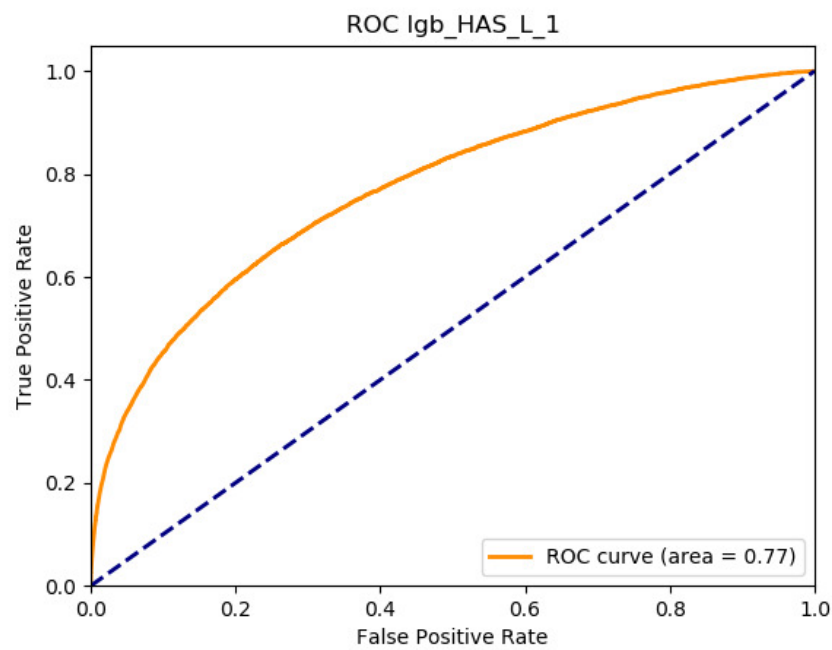
PRODUCT G ROC-AUC



PRODUCT H ROC-AUC



PRODUCT I ROC-AUC



PRODUCT L ROC-AUC